CENTRAL UNIVERSITY OF TECHNOLOGY

FACULTY OF ENGINEERING, BUILD ENVIRONMENT AND INFORMATION TECHNOLOGY

SUPACLEAN

LAUNDRY AND DRY-CLEANING MANAGEMENT SYSTEM

Year: 2026

PREPARED BY

GROUP M

NAME	STUDENT NUMBER
Masendeke CP	224043099
Mahlangu P	224014647
Maleke KT	222009259
Maema KE	224079714
Mojakwe KB	223032987
Moretlwe RK	221012635
Dasheka T	219007064
Leeu FD	223044569
Ntlati TT	224081629
Stallenberg BC	224081442

November 2026

Table of Contents

SUPACLEAN 1

Problem Statement 3

Introduction 3

Definitions, Acronyms and Abbreviations 4

Stakeholder and Organisation 4

Project Aim and Objectives 5

Project Scope 5

Stakeholders and Users 7

Current Process (refer to Appendix A for questions and answers asked to get this information) 7

Identified Problems 8
Overall Description and Product Perspective 9
Functional Requirements 9
Non-Functional Requirements 10
Integration Requirements 10
Requirements Summary Table 10
Technical Feasibility (refer to appendix B) 13
Operational Feasibility 13
Economic Feasibility 13
Schedule Feasibility 13
Legal Feasibility 13
Ethical Feasibility 14
Feasibility Conclusion 14
Risk Analysis (refer to appendix E for more risks) 15
((refer to appendix E for more risks) *23. Highest-Priority Risks (refer to Appendix E for the full risk register)* 15
Technical Designs 17
System Architecture designs 18
System Architecture Explanation 18
Use-Case Diagrams 19
Sequence diagram 1 Flutter 21
How to read the diagram 21
Explanation 22
Sequence diagram 2 ASP.NET 22
How to read the diagram 22
Explanation 22
Entity-Relationship Diagram (ERD) (refer to appendix F for matching case Diagram) 23
Project Plan Phase 2 26
GitHub Repository and Source Links 28
Original Design Source Links 28
Implementation and Final Prototype 31
Overview of the Implemented system 31
Implemented Technology Components 31
Requirements Implementation Status 32
Implemented Interfaces (Screenshots) 35
User Interface Designs and Final Prototypes 35
ASP.NET Web Application Screens 52

Implementation Changes and Remaining Limitations 63
Systems Workflows and Shared Processes 63
Final Prototype Summary 64
Project Plan – Phase 3 65
AI usage 67
Appendix Documents 67

Problem Statement

SupaClean currently uses physical pickup points and a ticket or docket system to manage customer laundry and dry-cleaning orders. Although the existing process supports the movement and processing of customer items, customer and order information is not connected through a single integrated digital platform.

As a result, customers have limited digital access to information about their orders and may need to rely on manual communication or physical interaction with pickup points. Employees also need to manage customer and order information throughout the different stages of the laundry and dry-cleaning process without a centralised digital platform.

The problem affects several users:

- Customers have limited access to digital order information and communication.
- Employees(Drivers) need to manage and communicate order information across different stages of the process.
- Administrators requires a better way to oversee relevant customer and order information.

The consequences of the current situation include increased reliance on manual communication, limited customer access to order information, difficulties communicating exceptional issues relating to customer items, reduced digital interaction with the business, and potential challenges in attracting and retaining customers.

Therefore, an improved digital solution is needed to provide customers and authorised employees with easier access to relevant information and improve communication surrounding the existing laundry and dry-cleaning process.

Introduction

The SupaClean Laundry and Dry-Cleaning Management System is a proposed digital solution for SupaClean, a laundry and dry-cleaning business that operates through multiple pickup and drop-off points. Customers currently take their clothing to these locations, where the items are received and each clipped with a waterproof paper of a certain colour depending on the location from which the laundry was collected from. The clothing is then collected by employees or drivers and transported to the main SupaClean facility for washing, dry cleaning, and pressing where necessary. Once the process is complete, the clothing is returned to the relevant pickup point for collection by the customer.

The project aims to develop a digital platform consisting of a Flutter mobile application for customers and an ASP.NET web application for employees , supported by a Supabase database. All administrative functions (including driver and administrator functions) are performed exclusively

through the ASP.NET web application; the Flutter mobile application is used by customers only. The system is intended to improve communication between SupaClean and its customers while providing authorised employees with a centralised way to manage customer and order information.

The main stakeholder for the project is Tim, the manager of SupaClean, who identified the need for a digital system that can improve customer interaction and assist the business in managing relevant order information.

Definitions, Acronyms and Abbreviations

Status-Based Tracking: Displaying an order's current recorded stage; this does not mean live GPS tracking of a driver.

Role-Based Access Control (RBAC): A method of restricting system functions and data according to a user's assigned role.

Order / Docket: A record of a customer's requested service and associated items; it may be linked to the existing ticket/docket process.

Depot / Pickup Point: A SupaClean location where customers leave or collect laundry and dry-cleaning items.

Customer: A person using the Flutter mobile application to browse services, place orders and view order information.

ASP.NET Web Application: The browser-based application used by authorised employees.

Flutter: UI framework used to build the customer mobile application.

Supabase: Backend platform used by SupaClean for authentication and PostgreSQL data storage.

SRS: Software Requirements Specification; a document describing the system's agreed requirements.

POPIA: Protection of Personal Information Act, South African legislation governing the processing of personal information.

NFR: Non-Functional Requirement; a quality, constraint or operating condition the system must satisfy.

MVP: Minimum Viable Product; the smallest version of the system that delivers the essential agreed functions.

FR: Functional Requirement; a capability or action the system must provide.

EFT: Electronic Funds Transfer; a bank-to-bank payment made outside the system.

ASP.NET: Microsoft web application framework used for the SupaClean staff web application.

Stakeholder and Organisation

Organisation: SupaClean

Type: Laundry and dry-cleaning business (refer to appendix C for evidence)

Stakeholder: Tim

Role: Manager

Contact: 051 430 4241 or 066 470 2253

SupaClean is a laundry and dry-cleaning business that uses different pickup/drop-off points. Customers leave their clothes at these locations, and employees collect the clothes and take them to the main facility for washing or dry cleaning. Once completed, the clothes are returned to the pickup point for the customer to collect them. We selected SupaClean because the stakeholder identified areas where the business could benefit from a digital system, particularly in communicating with customers and managing order information. (Appendix D)

Project Aim and Objectives

Project Aim

The aim of the project is to develop a mobile and web-based Laundry and Dry-Cleaning Management System that improves the way SupaClean manages customer and order information while providing customers with a more convenient way to interact with the business.

Project Objectives

The project aims to achieve the following objectives:

- Allow customers to view the laundry and dry-cleaning services offered by SupaClean.
- Allow customers to submit laundry or dry-cleaning requests through the mobile application.
- Allow customers to view relevant information about their orders.
- Allow authorised employees to view and manage customer orders through the ASP.NET web application dashboard.
- Allow admins to update the status of an order as it progresses through the process using the ASP.NET web application.
- Allow customers to receive important notifications relating to their orders.
- Provide a way for SupaClean to communicate relevant information when exceptional situations occur.
- Allow SupaClean to display specials and promotional information to customers.
- Customers can request home pickup.
- The system provides price estimates using service, clothing type/material, weight and applicable urgency/delivery charges.
- Provide administrators with access to relevant customer and order information through the ASP.NET web application.
- Restrict all administrative functions to the ASP.NET web application, using role-based access so that they cannot be accessed from the Flutter mobile application.

Project Scope

Project Deliverables

The project will deliver the following:

- A Flutter mobile application for SupaClean customers only. The mobile application will not contain administrative, employee functions.
- An ASP.NET web application for SupaClean authorised administrators and drivers. All administrative functions will be performed exclusively in this application.
- A Supabase database for storing and managing relevant system data.
- A Supabase authentication for authenticating users and employees, with role-based access so that administrative roles can only be used through the web application.
- Project documentation describing the analysis, design, development, and testing of the proposed system.

Acceptance Criteria

The primary acceptance criterion for the system is that:

A customer must be able to submit an order through the proposed system, and an authorized employee must be able to view and manage that order through the web application.

Additional functionality should support the objectives of allowing customers to access relevant order information and allowing employees to update and manage order statuses through the web application.

Inclusions

The following features are included in the project:

- Customer registration/login and cellphone number.
- Laundry and dry-cleaning service selection, including special orders such as carpet washing.
- Price estimation using service, clothing type/material, weight, urgency and applicable delivery fee.
- Home pickup request functionality within the supported service area.
- Unique bag/order number and transparent due date, price and terms information.
- Order status tracking through employee/driver status updates (captured in the ASP.NET web application) and customer notifications.
- Admin acceptance/rejection of special items, including leather-jacket assessment (web application only).
- Multi-depot selection and basic promotional/app-sharing features.
- The digital system will support and link to SupaClean's existing ticket or docket process. Replacing the existing operational ticket/docket process is outside the approved project scope.
- Administrative functions (management of customers, orders, services, pricing and delivery fees, special-item acceptance, employee/driver status updates and management oversight) available only in the ASP.NET web application, with role-based access control.
- Customer points screen and functions: Adds points when an order is delivered, can use points earned at payment and takes the discount off the total, returns points when order is cancelled and shows balance and points history

The proposed system is intended to support and improve the existing business process. It will provide digital customer and order information while the existing operational ticket/docket process remains in use.

Constraints

The project is subject to the following constraints:

- Limited time available during the semester or project period.
- Limited development experience within the project team.
- The system must be designed, developed, and completed within the required academic project period.

Assumptions

The project is based on the following assumptions:

- The stakeholder is expected to be available to provide feedback when required. SupaClean's existing ticket or docket process will continue to be used alongside the digital system.
- No online payment gateway will be implemented. The Flutter mobile application may display EFT/bank-transfer and cash-on-delivery options and instructions. EFT payments are completed through the customer's bank outside the application; authorised web users record or verify payment information and proof.
- Project requirements may be refined or changed based on stakeholder feedback.
- Authorised employees will use the ASP.NET web application, either on a computer or through a supported mobile browser. They will not use the Flutter customer mobile application.

Stakeholders and Users

Stakeholder or User	Role in the proposed system
External stakeholder	Manager
Mobile Application user	Customers (clients) only
Web application user	Drivers and Administrators
Other affected users	N/A

Current Process (refer to Appendix A for questions and answers asked to get this information)

A customer takes their clothes to one of the SupaClean pickup points. The employee receives the clothes and uses the company's existing ticket/docket system. An employee/driver then collects the clothes and takes them to the main facility. The clothes are washed or dry cleaned and pressed where necessary. Once completed, they are taken back to the pickup point where the customer collects them.

People involved

- Customers
- Pickup-point employees
- Drivers/collection employees
- Cleaning employees
- Management

Information used

- Customer details

- Ticket/docket number
- Items received
- Service requested
- Price
- Pickup point
- Order status
- Collection information

Current Process Diagram

Customer
↓
Pickup / Drop-off Point
↓
Driver
↓
Main SupaClean Facility
↓
Washing / Dry Cleaning
↓
Driver
↓
Pickup / Drop-off Point
↓
Customer

Identified Problems

The stakeholder identified that SupaClean could benefit from a digital system that improves communication with customers and provides better management of customer and order information.

Currently, customers mainly interact with SupaClean through physical pickup points and existing communication methods. There is no integrated digital platform that connects customers and authorized employees to information about the progress of an order.

Exceptional situations may also occur during the cleaning process. For example, an employee may discover a difficult stain or determine that a particular chemical could potentially damage an item. In these situations, SupaClean may need to communicate with the specific customer before continuing with the cleaning process.

Overall Description and Product Perspective

The system consists of a Flutter mobile application for customers, an ASP.NET web application for drivers and administrators, and Supabase as the shared backend. Customers can browse supported services, submit orders, request home pickup and delivery where available, receive price estimates, select supported depots or service areas, view recorded order statuses, choose an external payment method and receive notifications. Authorised staff use the web application to review customer information, manage orders and services, verify payment information or proof, assign drivers, and update collection, processing and delivery statuses. The system does not provide live driver GPS tracking or an online payment gateway.

Main product functions

Component	Main Functions
Flutter mobile application	Customers browse supported services, submit orders and home-pickup requests, view price estimates and order information, choose an external payment method, submit proof of payment where implemented, and receive notifications. No, employee functions are available in the mobile application.
Supabase backend	Provides shared authentication, PostgreSQL data storage and file storage for the Flutter and ASP.NET applications. It stores relevant customer, order, service and payment records. Access to records and administrative actions must be restricted according to the configured permissions and user roles.
ASP.NET Web application	All administrative functions, including customer/order/service/pricing management, special-item acceptance, payment verification, employee/driver status updates and management oversight, remain only in the ASP.NET web application. Drivers must confirm collections/deliveries and help in update order statuses through the ASP.NET web application; status updates are not live GPS tracking.

Functional Requirements

FR1: The customer must register/log in, provide a cellphone number, and browse services.

FR2: The customer must select a supported service, item type/material and weight, and request pickup and/or delivery.

FR3: The customer must receive a price estimate, due date, terms, payment-method information and order notifications.

FR4: Each bag/order must have a unique number that can be associated with the customer and order.

FR5: Administrators must manage customers/orders/services/pricing, accept or reject special items, and verify payment records or proof through ASP.NET web application.

FR6: Drivers must confirm collections/deliveries and update order statuses through the ASP.NET web application; status updates are not live GPS tracking.

FR7: The system must support special orders such as carpet washing and communicate exceptional cleaning issues to the customer.

FR8: The system must provide transparent price and status information; payments use external methods such as EFT/bank transfer or cash, and no online payment gateway is implemented

Non-Functional Requirements

NFR1: The system must support the versions agreed upon during the requirements analysis.

NFR2: The system must maintain data integrity by preventing orders from being matched with the wrong customer.

NFR3: The system must use a modular architecture to allow components to be easily decoupled and maintained.

NFR4: The system must require users to authenticate before accessing protected functionality and must apply role-based access control so that administrative functionality is available only to authorised staff roles.

NFR5: The system should be available during and after the stakeholder's operating hours.

NFR6: All administrative functionalities must be available only in the ASP.NET web application. The Flutter mobile application must not expose employee or management functions, and administrative roles must not be usable through it.

Integration Requirements

IR1: The application and web system must integrate with Supabase PostgreSQL to securely store and manage application data.

IR2: The application and web system must integrate with Google Maps for depot locations and supported pickup service areas, not live driver tracking.

IR3: The application and web system must integrate with Supabase Authentication for secure registration, login and authentication, including role-based access for customer, employee accounts.

IR4: The system must support secure manual payment record/proof verification; no online payment gateway is required.

Requirements Summary Table

ID	*Requirement*	*Type*	*Priority*	*Related stakeholder need*
FR1	Customers must register and log in, provide a cellphone number, and access their customer account.	Functional	Must Have	Improve customer access

FR2	Customers must browse supported services, select service and item details (including type/material and quantity or weight), and request pickup and/or delivery within supported areas.	Functional	Must Have	Improve service access and order placement
FR3	The system must provide an order price estimate, relevant due-date and terms information, and instructions for available external payment methods.	Functional	Must Have	Provide transparent pricing and payment information
FR4	Each bag/order must have a unique reference, and customers must be able to view their orders, recorded statuses, due dates and relevant notifications.	Functional	Must Have	Improve order traceability and transparency
FR5	Administrators must manage customers, orders, services and pricing, accept or reject special items, and review payment records or proof through the ASP.NET web application.	Functional	Must Have	Improve business management and verification
FR6	Drivers must confirm collections and deliveries and update relevant order statuses through the ASP.NET web application. These updates are status-based and do not provide live GPS tracking.	Functional	Must Have	Improve collection and delivery management
FR7	The system must support agreed special services and provide a way to communicate exceptional cleaning issues.	Functional	Must Have	Support special services and exception communication
FR8	The system must support external payment methods such as EFT/bank transfer or cash, with payment records or proof reviewed by authorised staff. No online payment gateway is implemented.	Functional	Must Have	Support external payment administration
NFR1	The system must support the software versions agreed during requirements analysis.	Non-functional	Should Have	Ensure compatibility

NFR2	The system must maintain data integrity and prevent orders from being associated with the wrong customer.	Non-functional	Must Have	Protect customer and order data
NFR3	The system must use a modular architecture to support maintainability.	Non-functional	Should Have	Improve maintainability
NFR4	Users must authenticate before accessing protected functionality, and role-based access must restrict administrative functions to authorised staff.	Non-functional	Must Have	Ensure security
NFR5	The system should be available during and beyond the business's operating hours, subject to hosting and service availability.	Non-functional	Should Have	Support customer access
NFR6	All administrative functionality must be available only through the ASP.NET web application and must not be exposed in the Flutter customer mobile application.	Non-functional	Must Have	Maintain the agreed application separation
IR1	The Flutter and ASP.NET applications must connect to Supabase PostgreSQL for authorised storage and retrieval of application data.	Integration	Must Have	Store and share application data
IR2	The system should integrate with Google Maps for depot locations and supported pickup service areas. This does not include live driver tracking.	Integration	Should Have	Locate depots and support service areas
IR3	The system must use Supabase Authentication for secure registration, login and role-based access.	Integration	Must Have	Secure user access
IR4	The system must support secure manual payment records and proof verification. An online payment gateway is not required.	Integration	Must Have	Support external payment verification

Technical Feasibility (refer to appendix B)

Selected: Feasible with changes

The proposed technologies—Flutter, ASP.NET and Supabase—are suitable for developing the system and do not require specialized hardware. However, the team has limited experience with some of these technologies, and using both mobile and web frameworks may increase the learning curve. The system does not directly replace or integrate with the existing paper-based ticket/docket process, so duplicate data entry or manual ticket-number linking may be required during the prototype phase. Live GPS tracking and online payment gateway integration are outside the approved scope. Service selection, price estimation, pickup and delivery requests, external payment-method selection, manual payment verification remain within scope. Restricting administrative functions to the ASP.NET web application keeps customer functions in Flutter and staff functions in one role-controlled interface.

Operational Feasibility

Selected: Feasible with changes

The proposed system is suitable for customers, drivers, and administrators because it improves order tracking, customer communication, and delivery management. However, employees may have to use both the existing paper-based system and the new digital system, which could increase their workload. Some users may also require training to use the system effectively. Because employees and drivers will use only the web application, it must be responsive and usable in a phone or tablet browser so that drivers can confirm collections and deliveries in the field. Therefore, the system is operationally feasible but may require a simpler employee workflow, training, and future integration with the existing system.

Economic Feasibility

Selected: Feasible

The system can be developed using mostly free technologies such as Flutter, ASP.NET, and Supabase, with minimal hardware and hosting costs during the prototype phase. The main cost is the development time of the team. Therefore, the project is economically feasible within the available resources, although additional costs may be required for hosting, hardware, and training if the system is continued after the prototype stage.

Schedule Feasibility

Selected: Feasible with changes

The project can be completed within the available semester if the team focuses on the minimum viable product (MVP). Developing both the Flutter mobile application and ASP.NET web application within a limited time may be challenging, especially with limited development experience. Priority should be given to the core customer workflows shown in the UI designs: service selection, order submission, pickup and delivery scheduling, price estimation, order-status viewing, notifications and the external payment-method workflow. Essential administrative and payment-verification functions should also be prioritised. Live GPS tracking and an online payment gateway remain excluded. Regular testing, stakeholder feedback and bug fixing should be included in the schedule.

Legal Feasibility

Selected: Feasible with changes

The project is legally feasible because it can comply with South African data protection requirements under POPIA. However, the system will handle customer and order information, so proper authentication, access control, secure data storage, and data minimisation must be implemented. Access to customer data and administrative functions will be limited to authorised staff roles in the web application. A basic privacy notice should also be provided, and sensitive information such as passwords and API keys must not be stored in public repositories.

Ethical Feasibility

Selected: Feasible with changes

The project is ethically feasible if customer and employee information is handled responsibly and fairly. Customers should be informed about how their information is used, while employees should only access information relevant to their roles. The app should remain optional during the prototype phase so that customers without smartphones or with limited digital skills are not excluded. Clear communication, accurate order information, and honest promotions should also be maintained.

Feasibility Conclusion

The proposed project is feasible with changes.

The main strengths of the project are the availability of suitable development technologies, low development costs, and the ability to address the stakeholder's need for improved laundry order and customer management.

The main limitations identified are the limited development experience of the group, the development of both a Flutter mobile application and an ASP.NET web application, the fixed semester deadline, and the need to comply with privacy and security requirements.

To ensure that the project can be completed within the semester, the group will prioritise the essential MVP functions, control the project scope, simplify or postpone lower-priority features, and allocate sufficient time for testing and stakeholder feedback.

Risk Analysis (refer to appendix E for more risks)

LIKELIHOOD: 1 = Unlikely • 2 = Possible • 3 = Likely

IMPACT: 1 = Low • 2 = Medium • 3 = High

RISK LEVEL: 1–3 Low • 4–6 Medium • 7–9 High

ID	RISK STATEMENT	CATEGORY	L	I	SCORE	LEVEL	MITIGATION ACTION	CONTIN
R01	Limited group experience with Supabase may delay development of the database and connections between application components.	Technical	3	3	9	High	Test basic Supabase functions early and allocate learning tasks to group members.	Simplif tested probler
R02	Difficulty linking the existing ticket/docket system with the proposed digital order may delay order management functionality.	Technical / Integration	3	3	9	High	Investigate the existing ticket system early and test how ticket numbers can be associated with digital orders.	Use au entry o the MV
R03	Flutter and ASP.NET development tasks may take longer than planned, leaving insufficient time for testing and final improvements.	Schedule / Technical	3	3	9	High	Break development into smaller tasks and test each component as it is completed.	Prioritis postpon
R04	The stakeholder may become unavailable or delay feedback, which could delay confirmation of requirements and system decisions.	Stakeholder	2	3	6	Medium	Arrange important stakeholder meetings and feedback dates in advance.	Use pr requir feedba

((refer to appendix E for more risks) *23. Highest-Priority Risks (refer to Appendix E for the full risk register)*

9 – High	1. R01 – Limited Supabase experience Supabase is the central database/backend. Test basic database and integration functions early.
9 – High	2. R02 – Existing ticket/docket integration The proposed system must work alongside SupaClean's existing ticket process. Investigate the system and test manual ticket-number linking.
9 – High	3. R03 – Flutter/ASP.NET development delays Delays could reduce testing time. Prioritise the core MVP and monitor progress against the schedule.

Requirements Change Log

Original Requirement	Change Made	Reason for Change

Marketing not considered	Sharing of app equals discount	To enhance marketing and access of application along with business
Original Requirement	**Change Made**	**Reason for Change**
App created for one store	App must integrate with various depots	The Dry Clean has many branches with one central factory
Only Dry Cleaning Services Considered	Other services provided by dry clean included e.g carpet cleaning	To create a centralised app for all services provided by the dry clean business
All customer delivery fee are the same	Customer not within a certain radius to the nearest depot will be provided discount while customer in a certain radius wont	This is to keep the depots operational and promote dropping off at depots where possible
Home pickup excluded	Home pickup request added; live GPS remains excluded	Add requested customer convenience without live tracking
Online payment gateway considered	Online payment removed	Use customer proof of payment verification instead of payment gateway
Price estimation (not specified)	Estimate price using service, clothing type/material, number of items/cloths, urgency and delivery fee	Give customers transparent estimation
Bag identification (not specified)	Unique number assigned to each bag/order	Improve bag and order traceability
Administration and employee/driver functions considered for both the mobile and web applications	All administration is performed only in the ASP.NET web application; the Flutter mobile application is for customers only	Keep sensitive administrative functions in one controlled interface, simplify the mobile application scope and reduce development effort

Refined Scope

Refined Inclusions (refer to appendix G , for initial stakeholder requirements and scope)

- Customer account registration/login, cellphone details.
- Supported services shown in the UI: wash & fold, dry cleaning, ironing, bedding & linen, shoe cleaning and other agreed/special services such as carpet washing
- Customer item selection with clothing type/material, quantity/weight and special instructions.

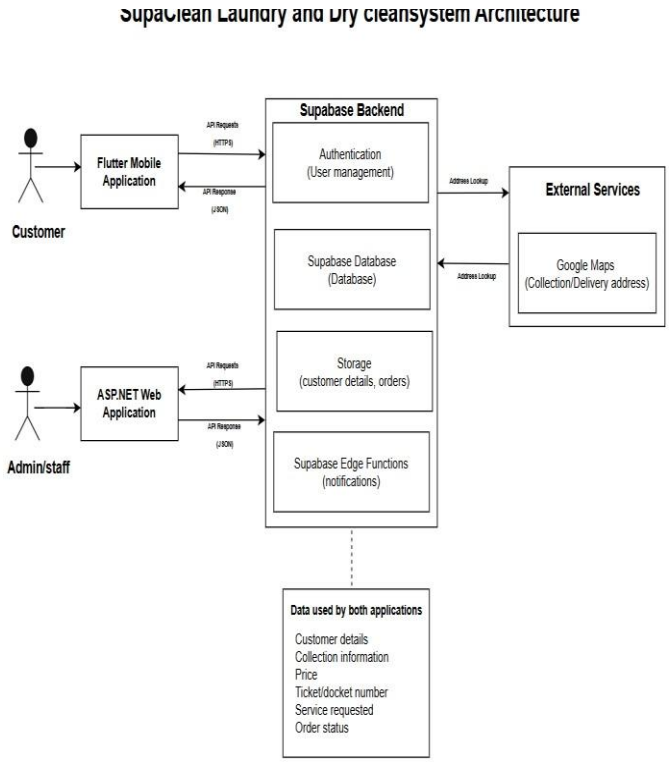
- Pickup and delivery requests with date/time slots and supported service areas.
- Price estimation using service, item details/weight, urgency and applicable pickup/delivery fees.
- Unique bag/order numbers, due dates, terms and transparent order information.
- Status-based order tracking and notifications; driver status updates are captured in the ASP.NET web application and there is no live GPS tracking.
- Multi-depot selection, promotions/app-sharing features and customer profile/settings.
- External payment-method selection/instructions (EFT/bank transfer or cash) with payment records/proof verified through the web application; no online payment gateway.
- All administrative functions, including customer/order/service/pricing management, special-item acceptance, payment verification, employee/driver status updates and management oversight, remain only in the ASP.NET web application.

Strict exclusions

Features, services, or deliverables not explicitly listed in the ‘in-Scope’ section of this document

Technical Designs

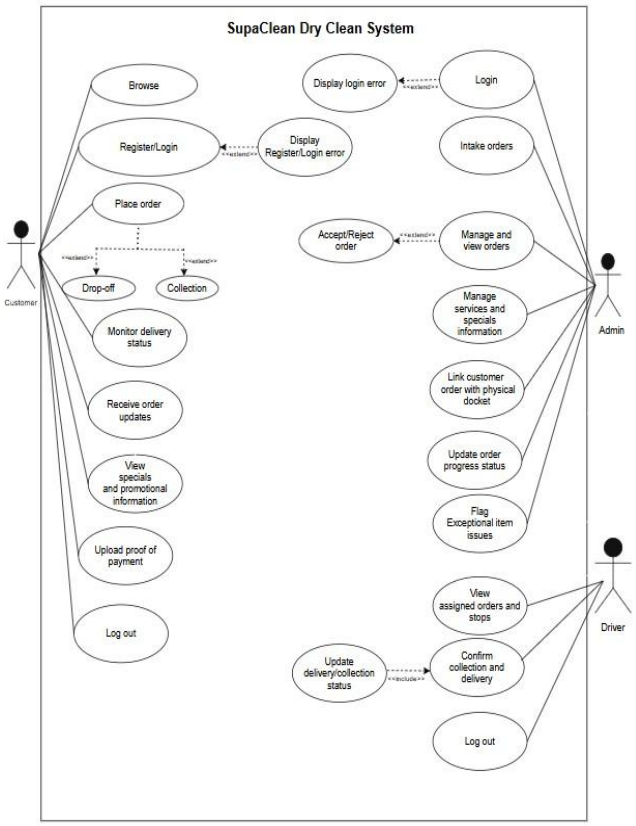
System Architecture designs



System Architecture Explanation

Customers use the Flutter mobile application to submit and view laundry and dry-cleaning orders as well as viewing the service catalogue. administrators, and drivers use the ASP.NET web application to manage orders, customers, and operational activities. Supabase provides authentication, database, storage, real-time updates for the system. Both Flutter and ASP.NET communicate with Supabase through

HTTPS and API requests and use the same shared backend and database. This allows the mobile and web applications to work together as one integrated SupaClean solution



Use-Case Diagrams

Use Case Explanation

A use case describes a function the system provides to an actor. The explanation follows the diagram as provided:

1. **Actors**

Customer: The customer uses the system to browse information, register or log in, place orders, upload proof of payment, monitor delivery status, receive order updates, view specials/promotional information, and log out.

Admin: The admin manages the operational side of the system. The diagram shows the admin logging in, taking in orders, managing and viewing orders, accepting/rejecting orders, managing services and specials information, linking customer orders with physical lockers, updating order progress, and nagging exceptional item issues

Driver: The driver handles assigned collection and delivery work. The diagram shows the driver viewing assigned orders and stops, confirming collection and delivery, updating delivery/collection Status, and logging out.

2. **Customer Use Cases Browse**: Allows a customer to browse the available information before or without carrying out an order.

Register/Login: Allows a customer to register for an account or log in. The diagram also provides a separate error use case for problems during this process. Place Order: Allows the customer to create a cleaning order. The diagram shows this use case including both Drop off and Collection, meaning these activities form part of the order process shown.

Drop off: Represents the customer dropping clothes/items off as part of the order process.

Collection: Represents the collection activity associated with the customer's order. Upload proof of payment: Allows the customer to upload proof Of payment (for example, a payment receipt or confirmation) for their order, so that payment can be confirmed.

Monitor delivery status: Allows the customer to monitor the status of delivery. Receive order updates: Allows the customer to receive updates concerning the order. View specials and promotional information: Allows the customer to view available specials and promotional information.

Log out: Allows the customer to end the logged-in session.

3. **Admin Use Cases Login:** Allows the admin to access the system. The diagram shows Display login error as an included behavior, meaning a login error can be displayed as part of the login process. Intake orders: Represents the driver taking orders into the system.

Manage and view orders: Allows the admin to manage and view orders. The diagram shows Accept/Reject order as included in this process.

Accept/Reject order: Allows the admin to accept or reject an order. Manage services and specials information: Allows the admin to manage service information and specials/promotional information. Link customer order with physical locker: Allows the admin to associate a customer order with a physical locker. Update order progress Status: Allows the admin to update the progress/status of an order.

Flag Exceptional item issues: Allows the admin to flag issues involving exceptional items.

4. **Driver Use Cases View assigned orders and stops:** Allows the driver to see the orders and stops assigned to them.

Confirm collection and delivery: Allows the driver to confirm that a collection or delivery has taken place. The diagram shows Update delivery/collection Status as included in this process.

Update delivery/collection Status: Updates the Status Of the relevant collection or delivery after confirmation. Log out: Allows the driver to end the logged-in session.

5. **Relationships in the Diagram include»:** An included use case part of the main use case's behavior. In the diagram, Place order includes Drop off and Collection; Login includes Display login error; Manage and view orders includes Accept/Reject order; and Confirm collection and delivery includes Update delivery/collection Status. •extend the diagram

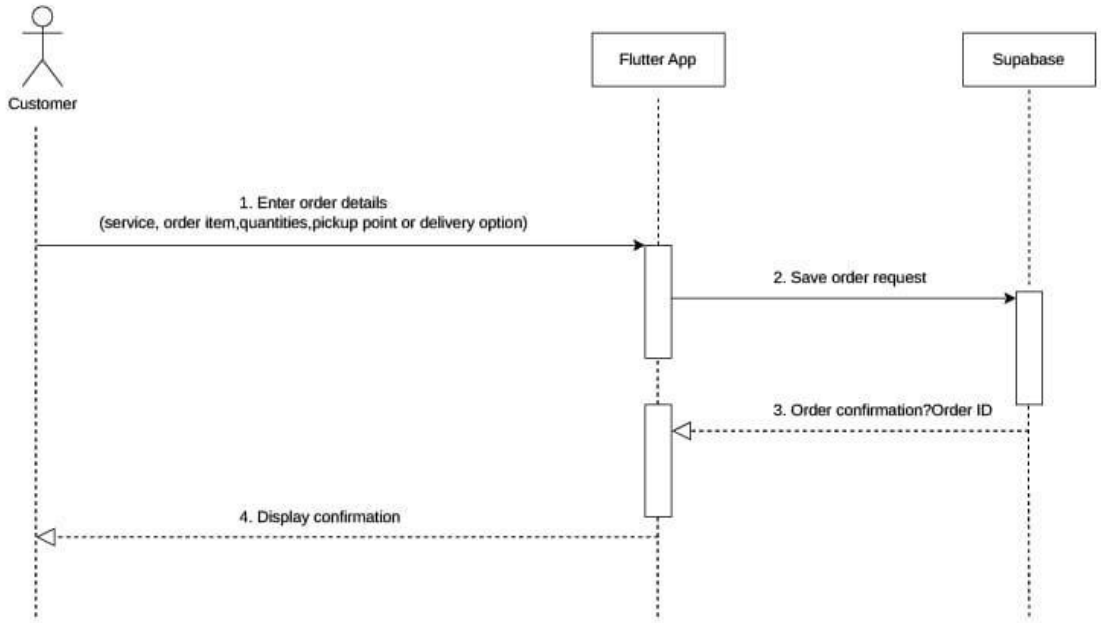
shows Display Register/Login error extending Register/Login. This represents an additional error behavior that can occur during the register/login process.

6. **Overall Flow Customer:**

A customer can browse, registering in, place an order, use the drop-off/collection activities associated with the Order, upload proof of payment, monitor delivery, receive updates, view specials/promotions, and log Out.

Admin: The admin logs in, handles incoming orders, manages/views them and accepts or rejects them, manages service/special information, links orders to physical lockers, updates order progress, and flags exceptional item issues.

Sequence diagram 1 Flutter



How to read the diagram

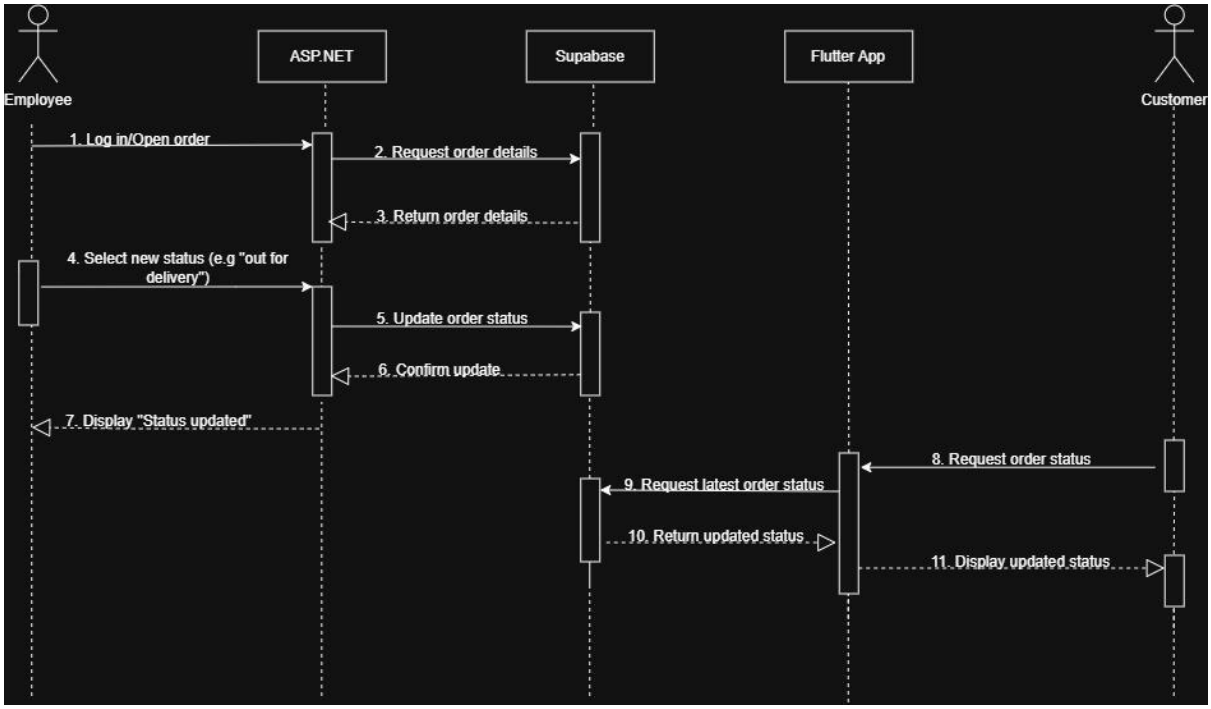
Read the diagram from top to bottom. Each vertical dotted line represents an actor or system involved in the process. The arrows show messages being sent between them. A solid arrow shows a request or action, while a dashed arrow shows information being returned. The numbered steps show the order in which the interaction happens.

Explanation

1. The Customer enters the order details in the Flutter App, including the service, order item, quantities, pickup point or delivery option.
2. The Flutter App sends the order request to Supabase so that the order can be saved.
3. Supabase returns an order confirmation and Order ID to the Flutter App.
4. The Flutter App displays the confirmation to the Customer.

Overall, the diagram shows how the customer creates an order in the Flutter App and how the app communicates with Supabase to save the order and receive its confirmation.

Sequence diagram 2 ASP.NET



How to read the diagram

Read the sequence from top to bottom. The vertical dotted lines represent the Employee, ASP.NET,

Supabase, Flutter App and Customer. The numbered arrows show which component communicates with another and the order in which this happens. Solid arrows represent requests or updates, while dashed arrows represent returned information or confirmations.

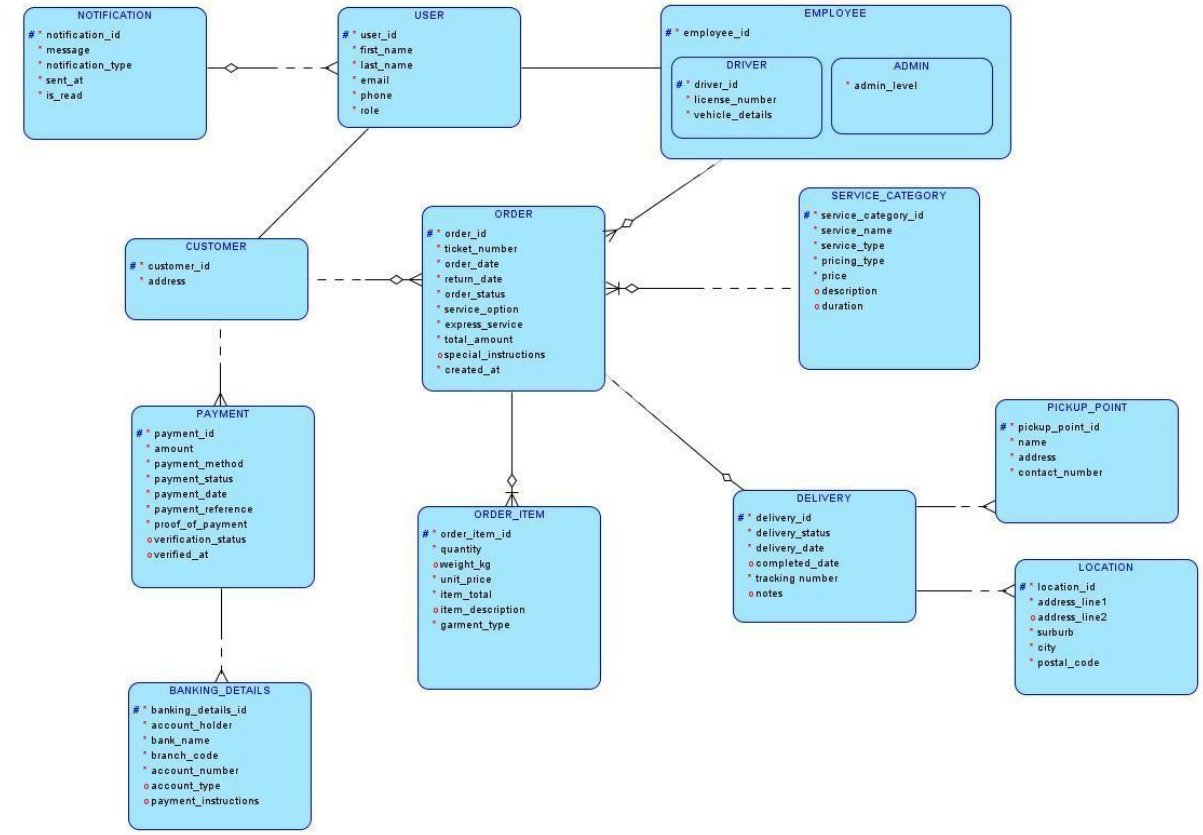
Explanation

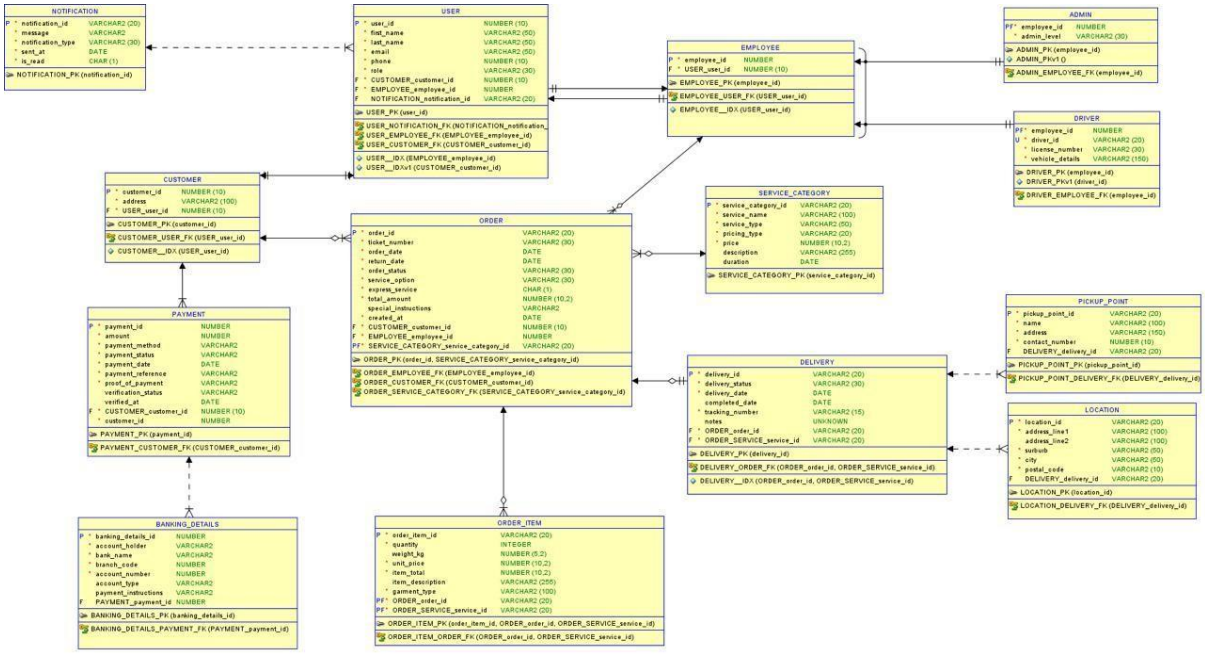
1. The Employee logs in or opens an order through ASP.NET.
2. ASP.NET requests the order details from the Supabase.
3. Supabase returns the order details to ASP.NET.
4. The admin selects a new order status, such as “out for delivery”.
5. ASP.NET sends the updated order status to Supabase.
6. Supabase confirms that the update was made.
7. ASP.NET displays “Status updated” to the Employee.
8. The Customer requests the order status through the Flutter App.

9. The Flutter App requests the latest order status from the Supabase.
10. Supabase returns the updated status to the Flutter App.
11. The Flutter App displays the updated status to the Customer.

Overall, the diagram shows how an employee updates an order status through ASP.NET and how the customer can later see the updated status through the Flutter App.

Entity-Relationship Diagram (ERD) (refer to appendix F for matching case Diagram)





How the system will work 1. **User registration and roles** ○ A person first registers as a **User**, with details such as name, email, phone, and role. ○ A user can be associated with a **Customer** account or an **Employee** account.

- Employees can have different responsibilities, mainly **Admin** or **Driver**.

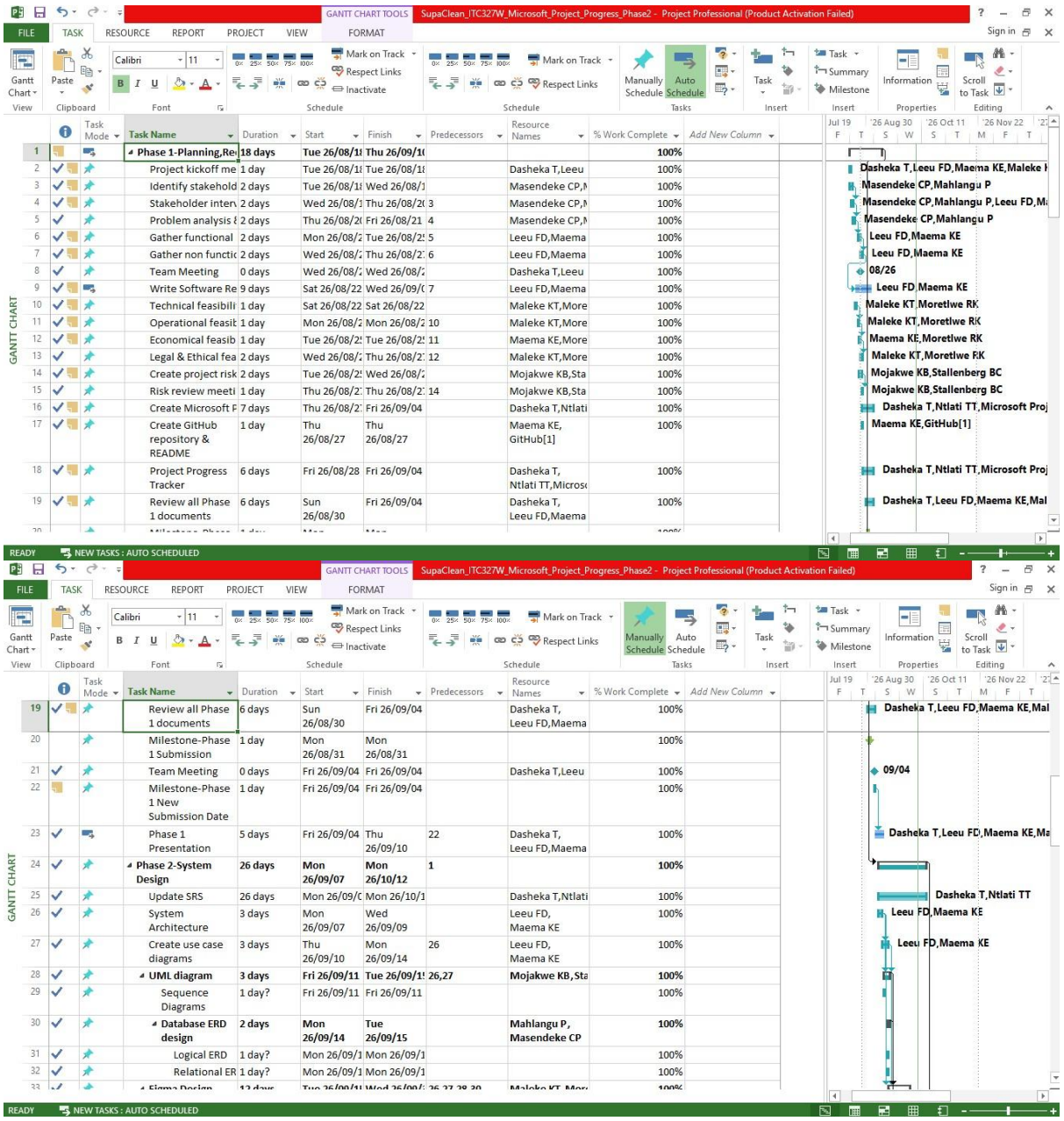
2. **Customer places an order** ○ A **Customer** provides an address and creates an **Order**.
   - The order stores information such as the ticket number, order date, return date, status, selected service, express-service option, total amount, and special instructions.
   - The customer can add one or more **Order Items**, containing details such as quantity, weight, price, description, and garment type.
3. **Services are selected** ○ Orders are associated with a **Service Category**.
   - A service category defines the type of service, pricing, description, and expected duration.
   - For example, different clothing-care or garment-processing services could be offered at different prices.
4. **Payment is recorded and verified** ○ The customer selects an available external payment method, such as EFT/bank transfer or cash on delivery. The system records the amount, method, status, date, reference and proof where applicable.
   - Authorised web users can review or verify payment information/proof and update the payment status.
   - Banking Details provide the information/instructions needed for an external EFT/bank-transfer payment; the system does not process an online payment gateway.
5. **Employees process the order** ○ Employees are responsible for managing orders.
   - **Admins** can manage the system, services, employees, orders, payments, etc.
   - **Drivers** have additional information such as their driver's ID, license number, and vehicle details.
6. **Delivery and pickup** ○ Once an order is ready, a **Delivery** record is created.
   - It tracks the delivery status, delivery date, completion date, tracking number, and notes.
   - **Pickup Points** and **Locations** store where items can be collected/delivered and the relevant address/contact information.
   - A driver can therefore be assigned to handle the delivery process.
7. **Notifications** ○ The **Notification** table allows the system to send updates to users, such as:

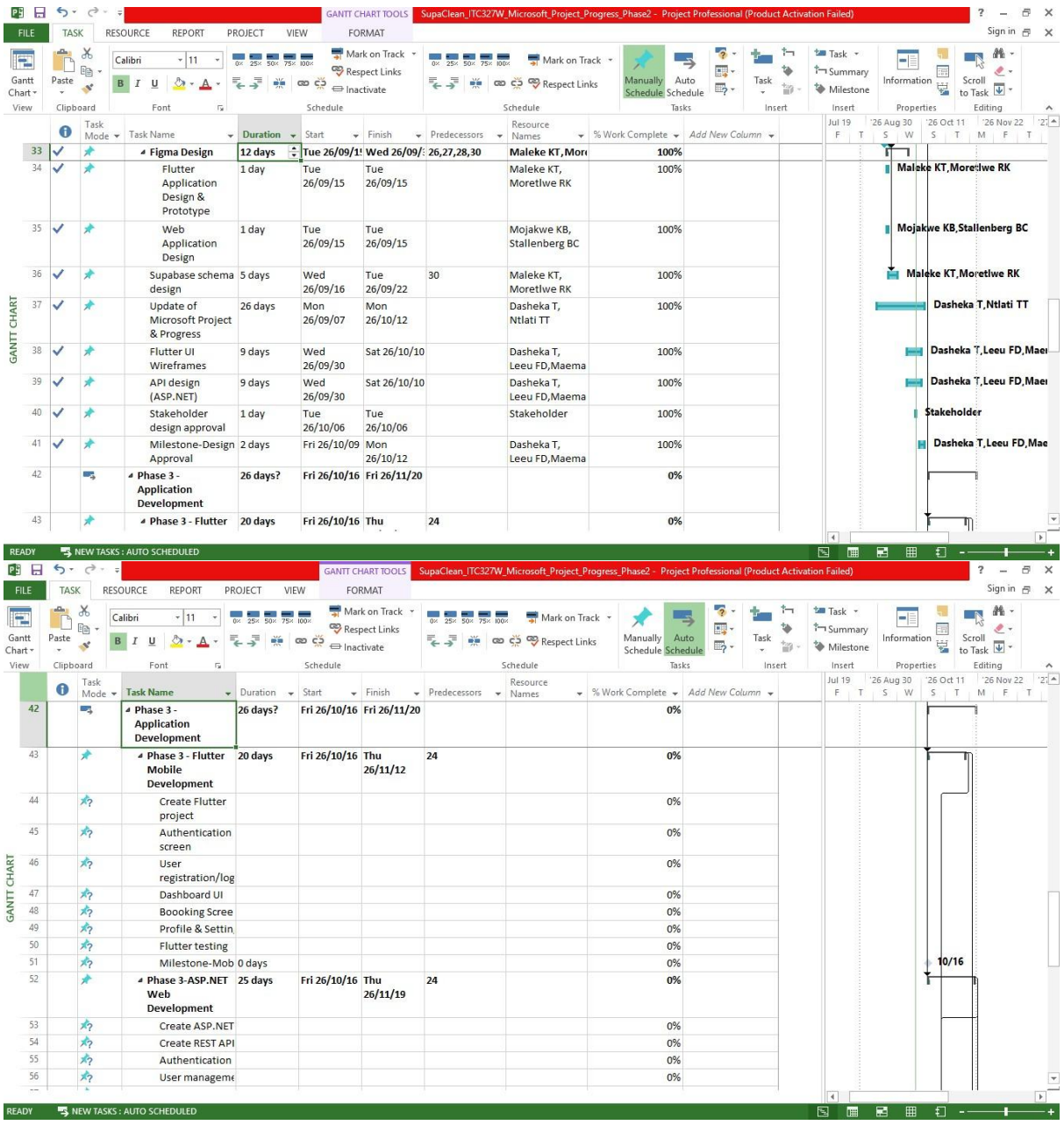
- Order received
- Payment confirmed
- Order being processed
- Order ready
- Out for delivery
- Order delivered ○ The is read field allows the system to track whether the user has viewed a notification.

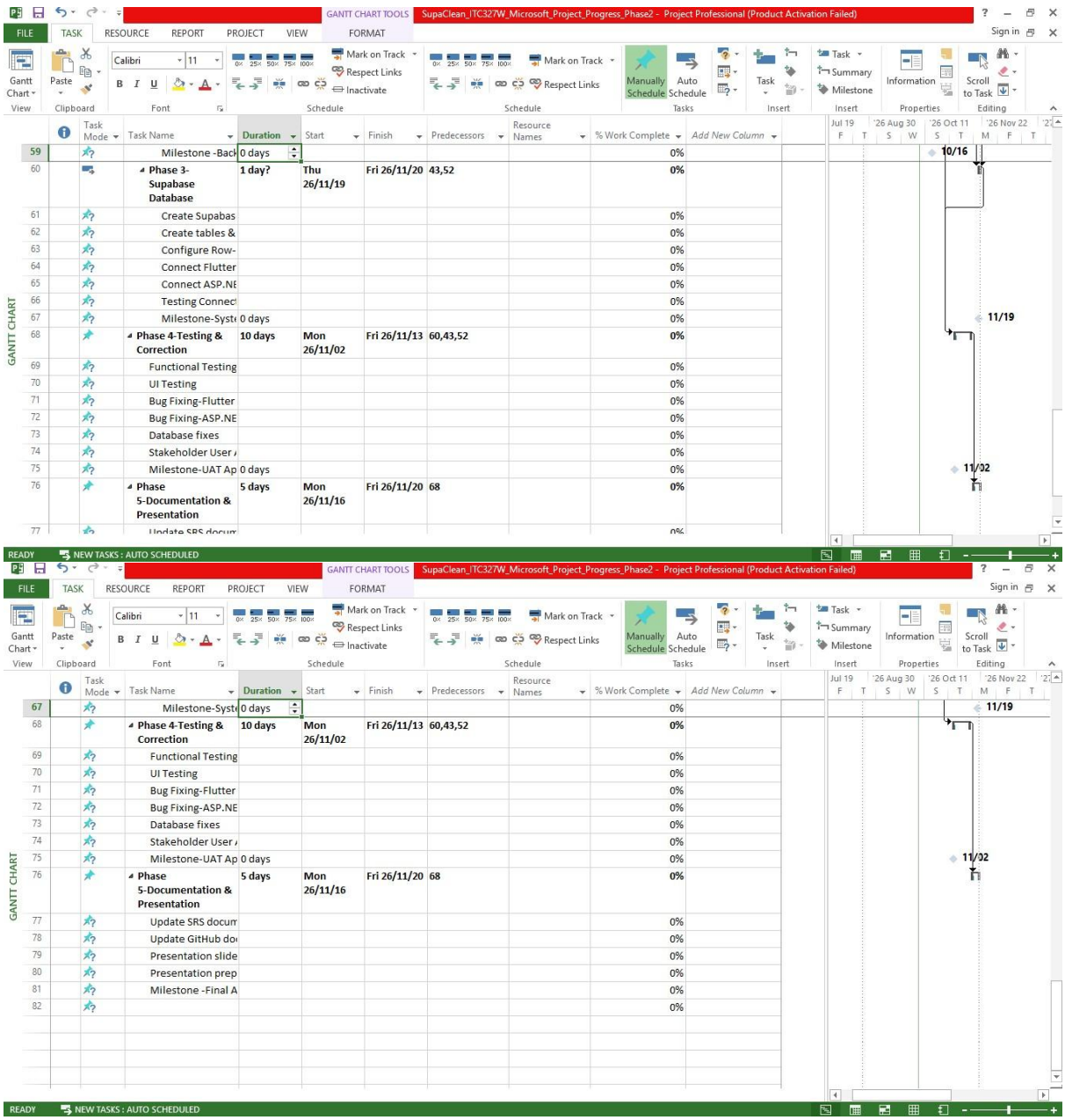
Overall workflow

Updated overall workflow: User registers → Customer places order → Selects supported service → Adds garments/items → Selects pickup/delivery option → Selects an external payment method and completes payment outside the gateway → Payment record/proof is verified by authorised web staff → Employee processes order → Driver handles pickup/delivery → Customer receives notifications → Order is completed.

Project Plan Phase 2




GANTT CHART TOOLS
SupaClean_ITC327W_Microsoft_Project_Progress_Phase2 - Project Professional (Product Activation Failed)
FILE
TASK
RESOURCE
REPORT
PROJECT
VIEW
FORMAT
Sign in
Gantt Chart
Paste
Calibri
11
Mark on Track
Respect Links
Inactivate
Manually Schedule
Auto Schedule
Task
Summary
Milestone
Information
Scroll to Task
View
Clipboard
Font
Schedule
Tasks
Insert
Properties
Editing
Task Mode
Task Name
Duration
Start
Finish
Predecessors
Resource Names
% Work Complete
Add New Column
33 Figma Design 12 days Tue 26/09/15 Wed 26/09/ 26,27,28,30 Maleke KT,Mor 100%
34 Flutter Application Design & Prototype 1 day Tue 26/09/15 Tue 26/09/15 Maleke KT, Moretlwe RK 100%
35 Web Application Design 1 day Tue 26/09/15 Tue 26/09/15 Mojakwe KB, Stallenberg BC 100%
36 Supabase schema design 5 days Wed 26/09/16 Tue 26/09/22 30 Maleke KT, Moretlwe RK 100%
37 Update of Microsoft Project & Progress 26 days Mon 26/09/07 Mon 26/10/12 Dasheka T, Ntlati TT 100%
38 Flutter UI Wireframes 9 days Wed 26/09/30 Sat 26/10/10 Dasheka T, Leeu FD,Maema 100%
39 API design (ASP.NET) 9 days Wed 26/09/30 Sat 26/10/10 Dasheka T, Leeu FD,Maema 100%
40 Stakeholder design approval 1 day Tue 26/10/06 Tue 26/10/06 Stakeholder 100%
41 Milestone-Design Approval 2 days Fri 26/10/09 Mon 26/10/12 Dasheka T, Leeu FD,Maema 100%
42 Phase 3 - Application Development 26 days? Fri 26/10/16 Fri 26/11/20 0%
43 Phase 3 - Flutter 20 days Fri 26/10/16 Thu 24 0%
Jul 19
'26 Aug 30
'26 Oct 11
'26 Nov 22
Maleke KT,Moretlwe RK
Mojakwe KB,Stallenberg BC
Maleke KT,Moretlwe RK
Dasheka T,Ntlati TT
Stakeholder
GANTT CHART
READY
NEW TASKS : AUTO SCHEDULED
42 Phase 3 - Application Development 26 days? Fri 26/10/16 Fri 26/11/20 0%
43 Phase 3 - Flutter Mobile Development 20 days Fri 26/10/16 Thu 26/11/12 24 0%
44 Create Flutter project 0%
45 Authentication screen 0%
46 User registration/log 0%
47 Dashboard UI 0%
48 Boooking Scree 0%
49 Profile & Settin 0%
50 Flutter testing 0%
51 Milestone-Mob 0 days 0%
52 Phase 3-ASP.NET Web Development 25 days Fri 26/10/16 Thu 26/11/19 24 0%
53 Create ASP.NET 0%
54 Create REST API 0%
55 Authentication 0%
56 User manageme 0%
10/16



GitHub Repository and Source Links

The official GitHub repository for the SupaClean Laundry & Dry-Cleaning Management System contains the project documentation and development materials.
https://github.com/Rethabile053/SupaClean.git– GitHub Repository

Original Design Source Links

Designs	Tools	Links

System Architecture	Draw.io	https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=_blank&layers=1&nav=1&title=Untitled%20Diagram.drawio&dark=auto#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1XqqlXLPAQ95Rfrx_avnEkedu4mxzd4Z3%26export%3Ddownload#%7B%22pageId%22%3A%22kkbFF_4SpVCPfKQWCokU%22%7D
Use Case Diagram	Draw.io	https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=_blank&layers=1&nav=1&title=Untitled%20Diagram.drawio&dark=auto#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D1XqqlXLPAQ95Rfrx_avnEkedu4mxzd4Z3%26export%3Ddownload#%7B%22pageId%22%3A%22n9kth-eKyPKHKWB2Ja-N%22%7D
Sequence diagram (ASP)	Draw.io	https://tinyurl.com/544bu4xv
Sequence Diagram (Flutter)	Draw.io	https://tinyurl.com/5fvvwn8n
Figma wireframe (Flutter)	Figma	https://www.figma.com/design/zjqgdakdIb2oFFJ4VQgwqI/SupaClean-group-M?nodehttps://www.figma.com/design/zjqgdakdIb2oFFJ4VQgwqI/SupaClean-group-M?node-id=0-1&t=UBJKCnPmMqEUqezu-1id=0-1&t=UBJKCnPmMqEUqezu-1

Figma wireframe	Figma	https://www.figma.com/proto/o5isMSBlZ8agg8lQCThLJp/SupaClean-ASP.Nethttps://www.figma.com/proto/o5isMSBlZ8agg8lQCThLJp/SupaClean-ASP.Net-screens?node-id=3-748&p=f&t=yqpijGY8m9jWjJPL-1&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=3%3A748screens?node-id=3-748&p=f&t=yqpijGY8m9jWjJPL-1&scaling=min-zoom&contenthttps://www.figma.com/proto/o5isMSBlZ8agg8lQCThLJp/SupaClean-ASP.Net-screens?node-id=3-748&p=f&t=yqpijGY8m9jWjJPL-1&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=3%3A748scaling=fixed&page-id=0%3A1&starting-point-node-id=3%3A748
(ASP)		
ERD	Data Modeler	https://drive.google.com/file/d/1vrVZIrOHD-6Uls9ZhqSDEGA0QAiMzUbZ/view?usp=sharing

Implementation and Final Prototype

Overview of the Implemented system

The SupaClean Laundry and Dry-Cleaning Management System is being implemented as a digital solution to improve customer interaction and the management of laundry and dry-cleaning orders.

The system consists of three main components: a Flutter mobile application for customers, an ASP.NET web application for employees, and Supabase as the shared backend and database platform.

The Flutter mobile application enables customers to access supported services, submit orders, request pickup and delivery, view order information and monitor recorded order statuses. Customers can submit proof of payment for external payments where this functionality is implemented.

The ASP.NET web application provides authorised drivers and administrators with the administrative functions required to manage customers, orders, services, pricing, payment verification and order-status updates.

Supabase provides the shared data storage and authentication services that connect the mobile and web applications. Changes made by authorised users through the web application are stored in the backend and can subsequently be retrieved and displayed by the customer mobile application.

This section documents the implemented prototype, the requirements addressed by the implementation, the evidence of working functions, approved changes and any remaining limitations.

Implemented Technology Components

Components	Technology	Implementation Description
Customer mobile application	Flutter and Dart programming language (Visual Studio Code)	Customer-facing screens and supported order functions.
Administrative web application	ASP.NET and C# programming language (Visual Studio and Visual Studio Code)	Staff login, dashboards and administrative order-management functions.
Shared backend	Supabase	Shared database and backend services used by both applications
Authentication	Supabase Authentication	Registration, login and access control.
Database	Supabase PostgreSQL	Storage of customer, order, service and related records.
File Storage	Supabase Storage	Storage of uploaded proof of payments

Brief description of components communication: The customer uses the Flutter mobile application to submit an order. The application sends the order information to Supabase, where it is stored and associated with the relevant customer. An authorised employee subsequently accesses the order through the ASP.NET web application and updates its status. The update is saved to Supabase, allowing the Flutter application to retrieve and display the latest recorded status.

This workflow demonstrates the intended integration between the customer application, administrative web application and shared backend. The extent to which each step is operational is documented in the implementation status table and supporting screenshots.

Requirements Implementation Status

ID	*Requirement / Functionality*	*Type*	*Status*
FR1	Customer registration, login and cellphone details	Functional	Reported as working; demonstrated
FR2	Service selection, item details and pickup/delivery requests	Functional	Reported as working; verified the complete workflow
FR3	Price estimate, due-date/terms information and external payment instructions	Functional	Reported as working; verified the calculation and displayed information
FR4	Unique order/bag reference, order viewing, status information and notifications	Functional	Reported as working; verified end-to-end status and notifications
FR5	Administrator management of customers, orders, services, pricing, special items and payment verification	Functional	Reported as working; verified each administrative function
FR6	Employee/driver collection, delivery and order-status updates through ASP.NET	Functional	Reported as working; verified role permissions and status flow

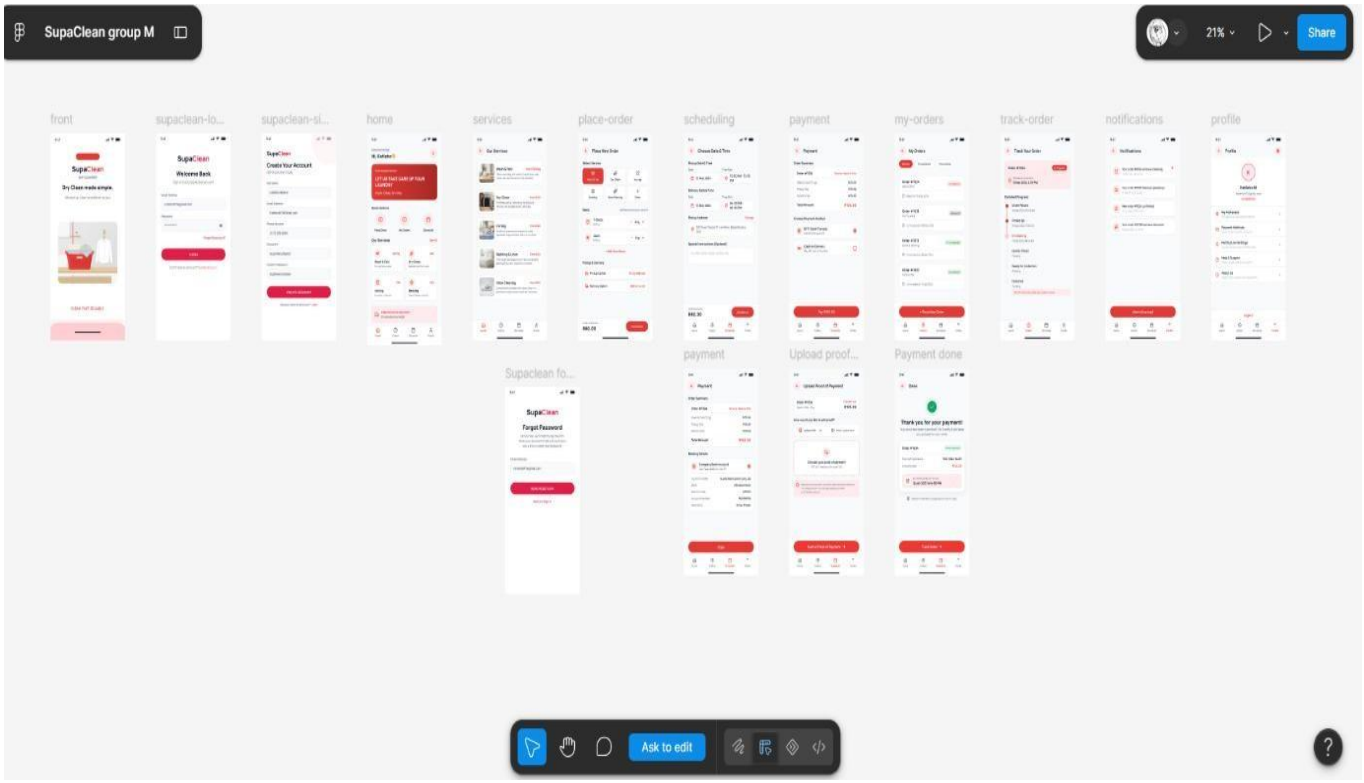
FR7	Special services and exception communication	Functional	Reported as working; verified using the demonstrated workflows
FR8	External payment recording and proof verification; no online payment gateway	Functional	Reported as working; verified using EFT and cash workflows
NFR1	Support for agreed software versions	Non-functional	Reported as working; document tested versions
NFR2	Prevent orders from being associated with the wrong customer	Non-functional	Reported as working; tested
NFR3	Modular architecture	Non-functional	Reported as working; support with design/code evidence
NFR4	Authentication and role-based access control	Non-functional	Reported as working; tested each role
NFR5	Availability during and beyond operating hours	Non-functional	Reported as working; has be verified in the deployed environment
NFR6	Administrative functions restricted to ASP.NET	Non-functional	Reported as working; verified application permissions
IR1	Supabase PostgreSQL integration	Integration	Reported as working; has integration evidence

IR2	Google Maps integration for depot/service-area information	Integration	Reported as working; has been tested
IR3	Supabase Authentication and role-based access	Integration	Reported as working; authentication evidence
IR4	Manual payment records and proof verification; no gateway	Integration	Reported as working; verified EFT and cash workflows

Implemented Interfaces (Screenshots)

User Interface Designs and Final Prototypes

Mobile Flutter Designs



1. Overall application workflow

1.1 Splash Screen

Introduces the SupaClean brand and services.

1.2 Sign Up / Login

Customers create an account or sign in.

1.3 Home Screen

Customers view services, prices and quick actions.

1.4 Place Order → Schedule Pickup and Delivery

Select laundry items, specify quantities, choose dates and addresses.

1.5 Payment

Review the total and choose a payment method.

1.6 Order Tracking and Notifications

Follow order progress, receive updates and check completion.

1.7 The My Orders, Services, Notifications and Profile screens support the main workflow by allowing customers to manage existing orders, explore services, receive updates and maintain their account information.

2. Explanation of each screen

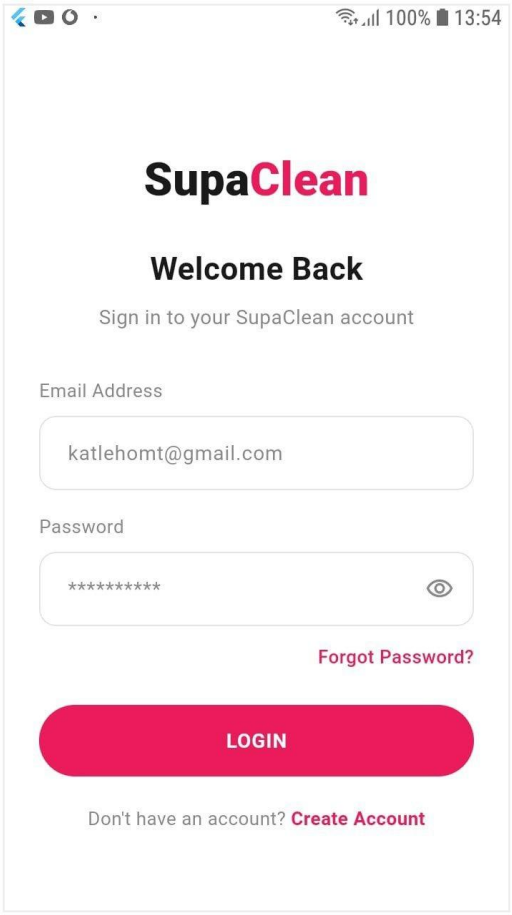
1. Splash Screen

Displays the SupaClean name, dry-cleaning identity and slogan. It introduces the application before the customer proceeds to registration or login.

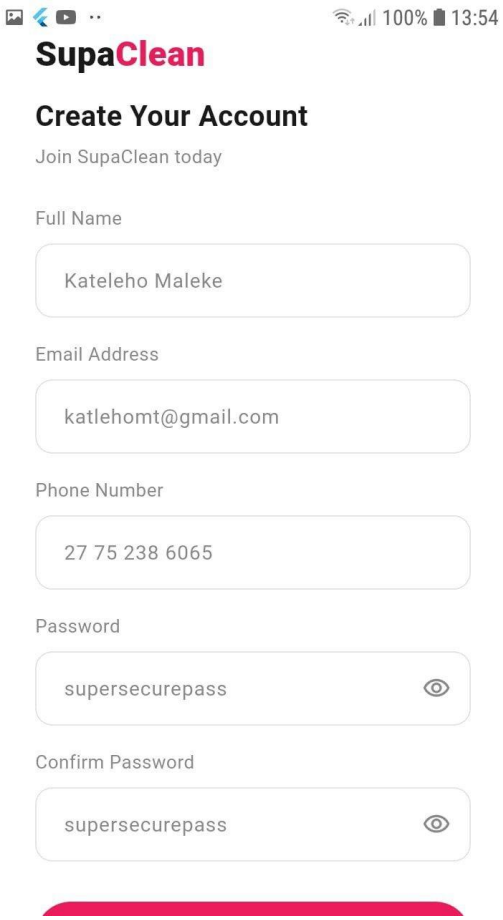
2. Sign Up and Login Screens

New customers enter their name, email address, phone number and password to create an account. Existing customers enter their email and password to sign in. The login screen also provides a forgot-password option.

Flutter screen — Login / Welcome Back



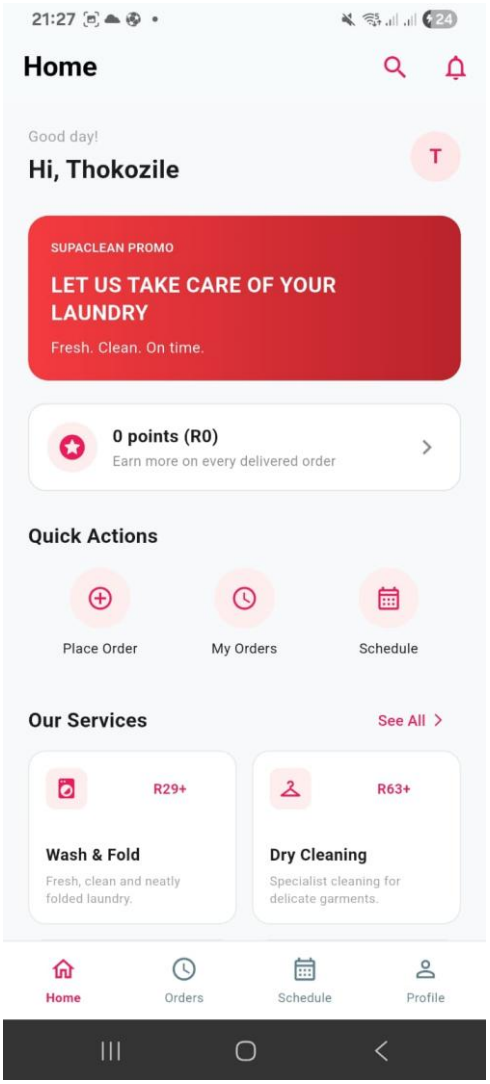
Flutter screen — Create Account



3. Home Screen

Acts as the main dashboard after login. It displays a welcome message, promotions, available services, prices and quick actions such as Place Order, My Orders and Schedule.

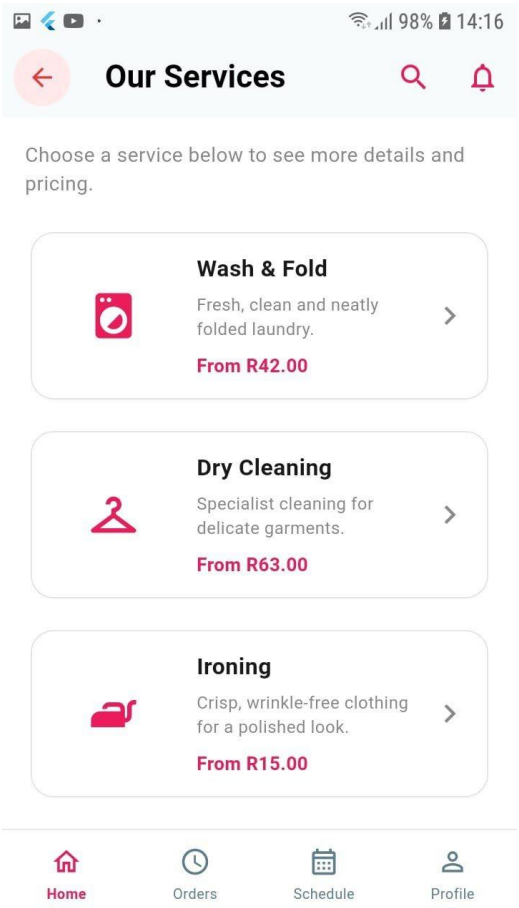
Flutter screen — Home



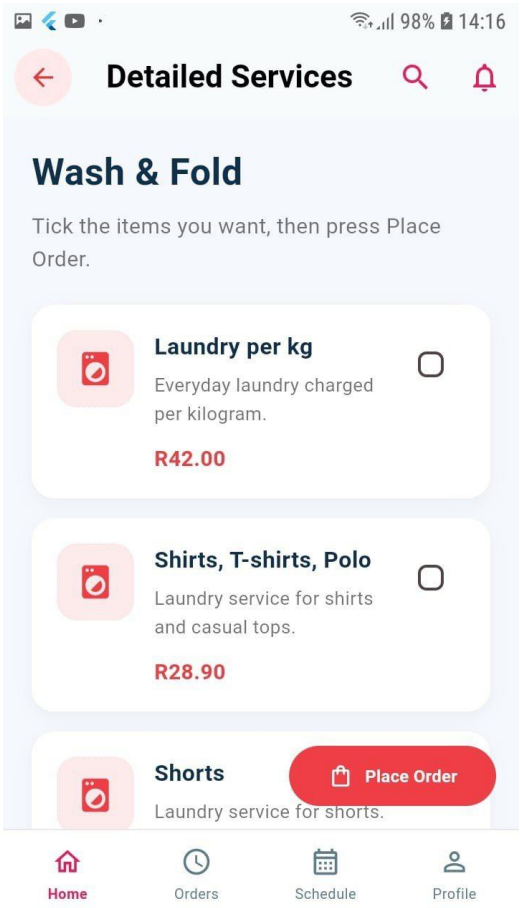
4. Our Services Screen

Displays the available services, including Wash & Fold, Dry Cleaning, Ironing, Bedding and Linen, and Shoe Cleaning. Customers can compare services and their starting prices before placing an order.

Flutter screen — Our Services



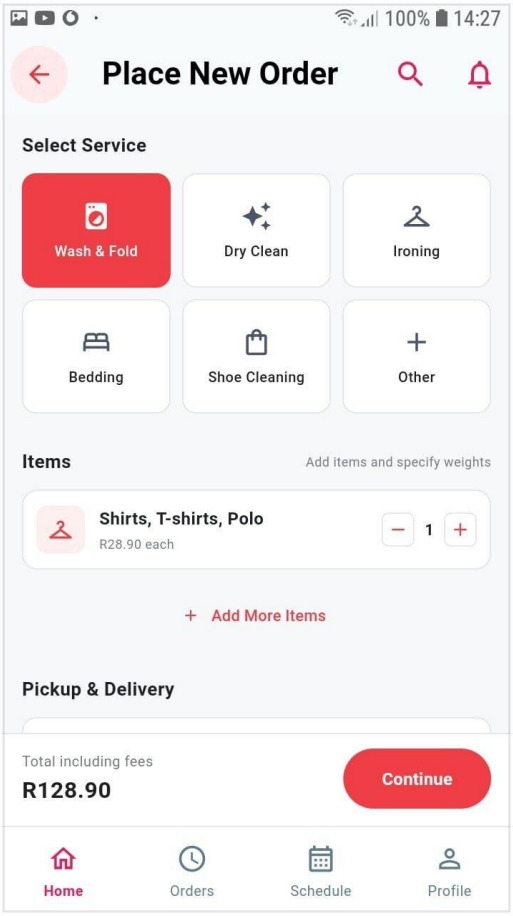
Flutter screen — Detailed Services



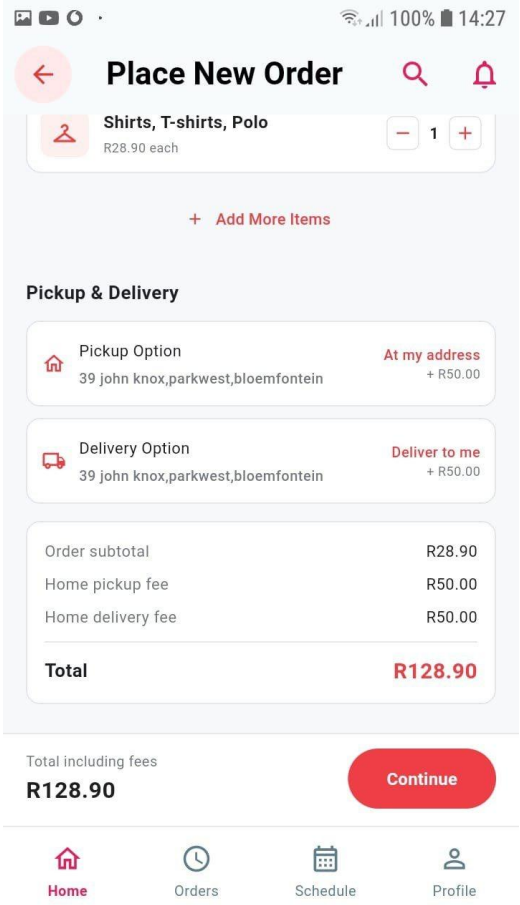
5. Place New Order Screen

The customer selects a service, adds laundry items, specifies quantities or weights, and chooses pickup and delivery options. The application calculates and displays an order subtotal before the customer continues.

Flutter screen — Place New Order



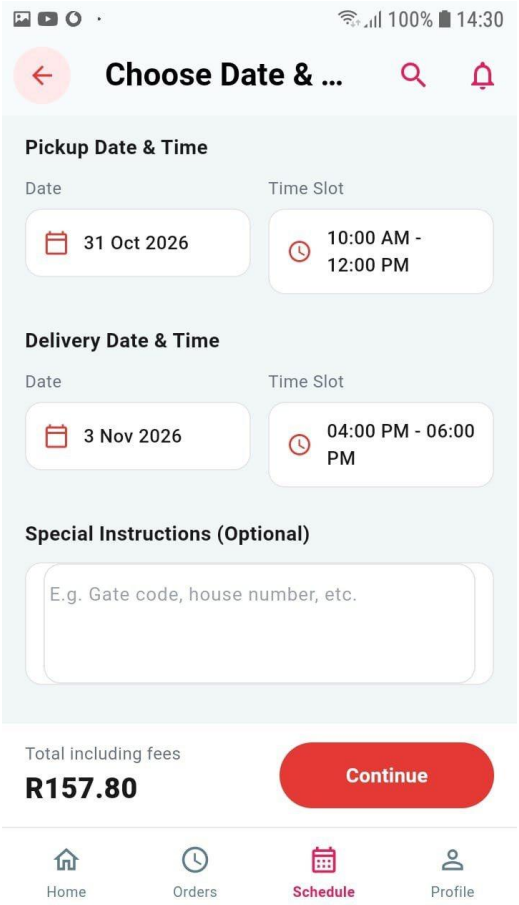
Flutter screen — Order Items / Selection



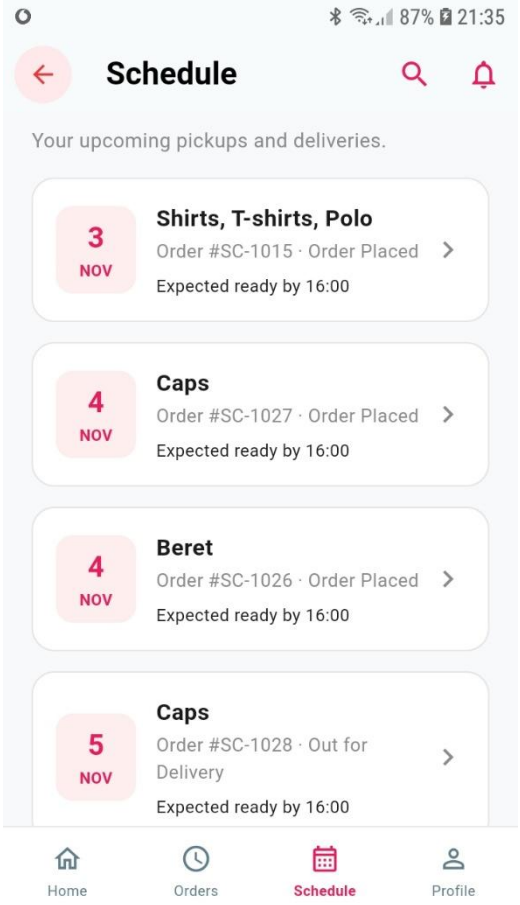
6. Scheduling Screen

Collects the pickup date and time, delivery date and time, pickup address and optional special instructions. The customer reviews the order subtotal and continues to payment.

Flutter screen — Choose Date and Time



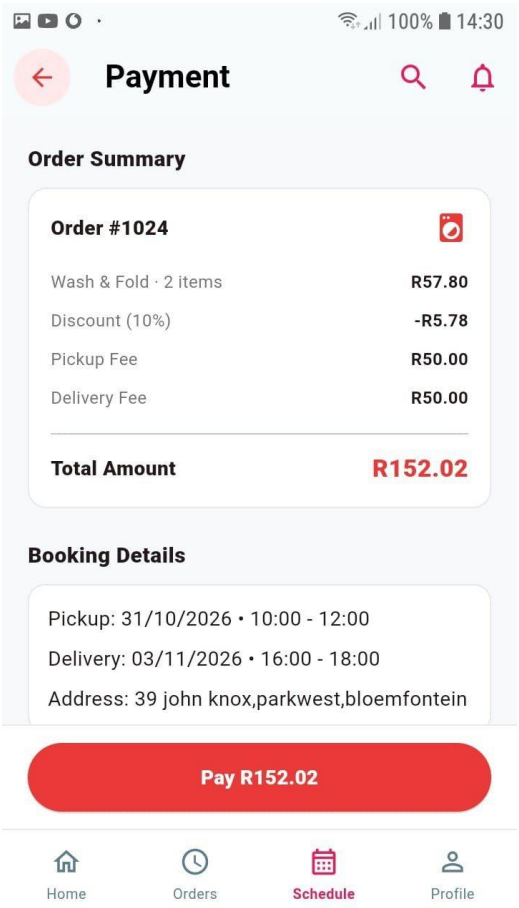
Flutter screen — Scheduling Details



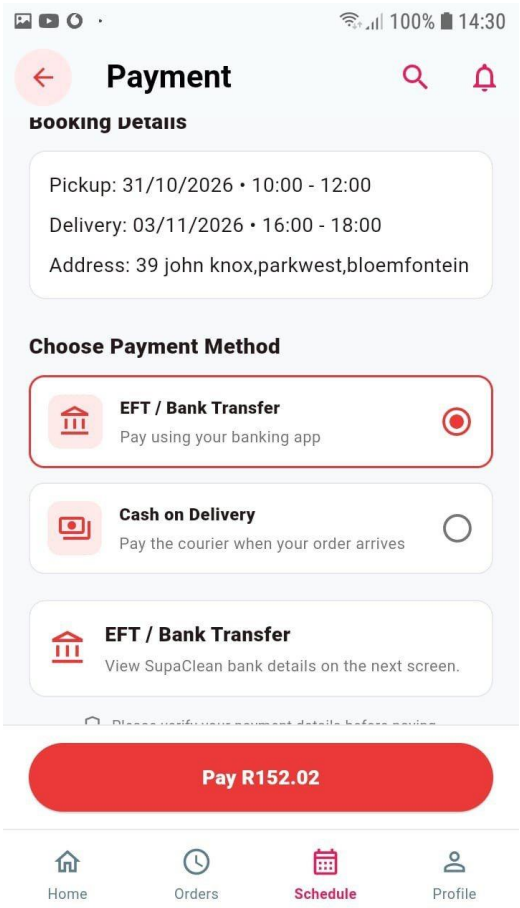
7. Payment Screen

Shows the order summary, service charges, pickup fee, delivery fee and total amount. Customers can choose EFT/bank transfer or cash on delivery, depending on the available payment options.

Flutter screen — Payment Summary



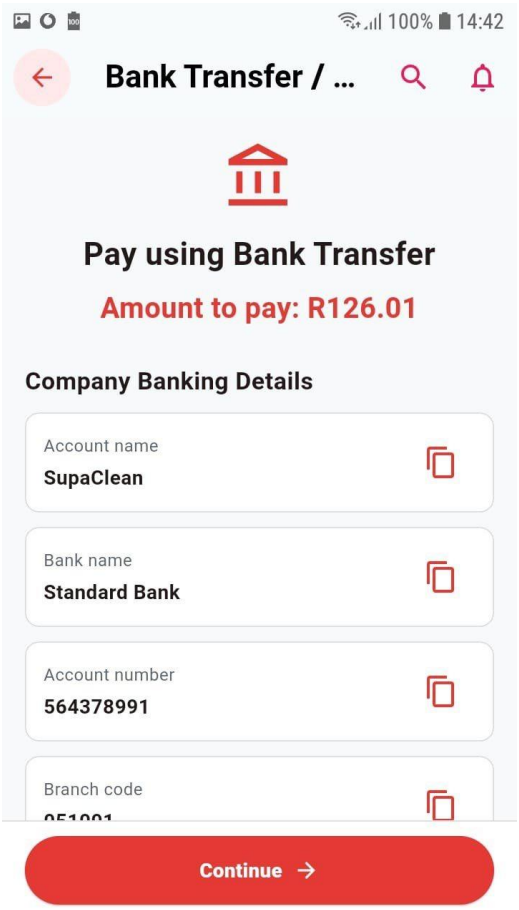
Flutter screen — Payment Method



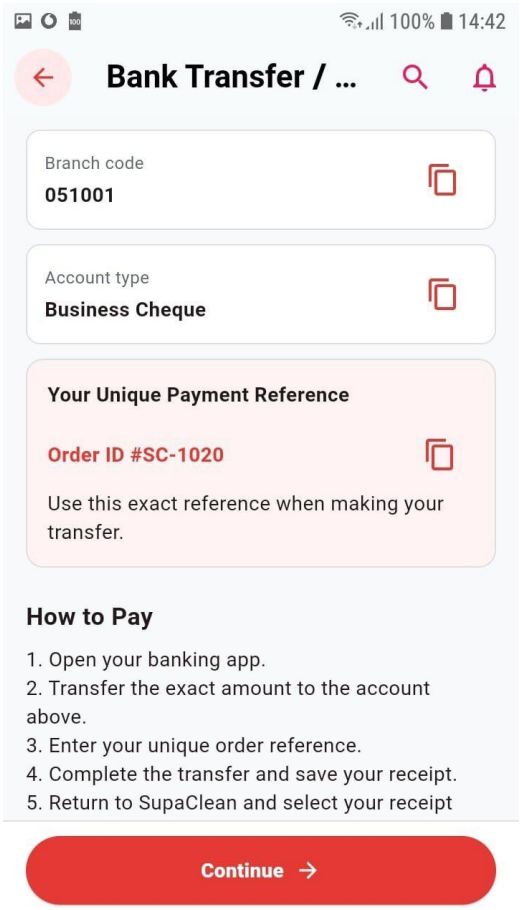
8. Banking Details Screen

Displays the company's bank account information and the order-specific payment reference. Customers can copy the details to make an EFT payment.

Flutter screen — Bank Transfer Instructions



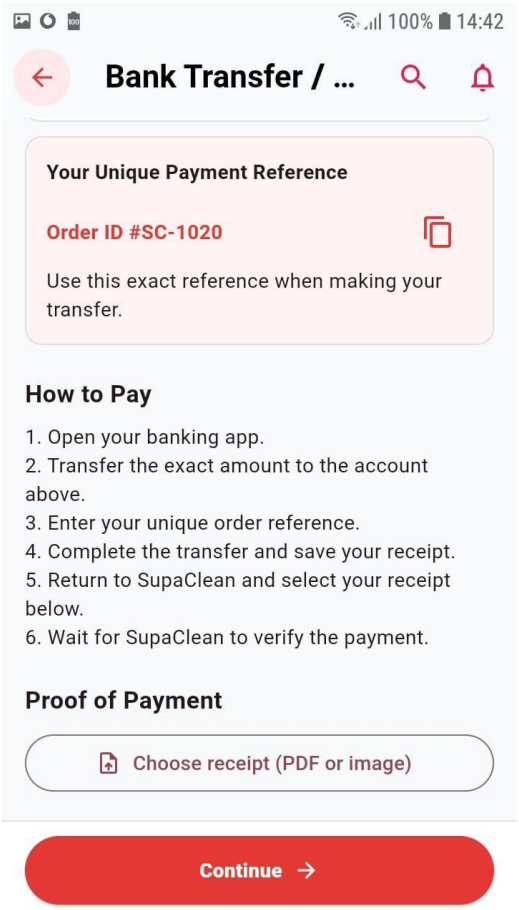
Flutter screen — Bank Transfer Details



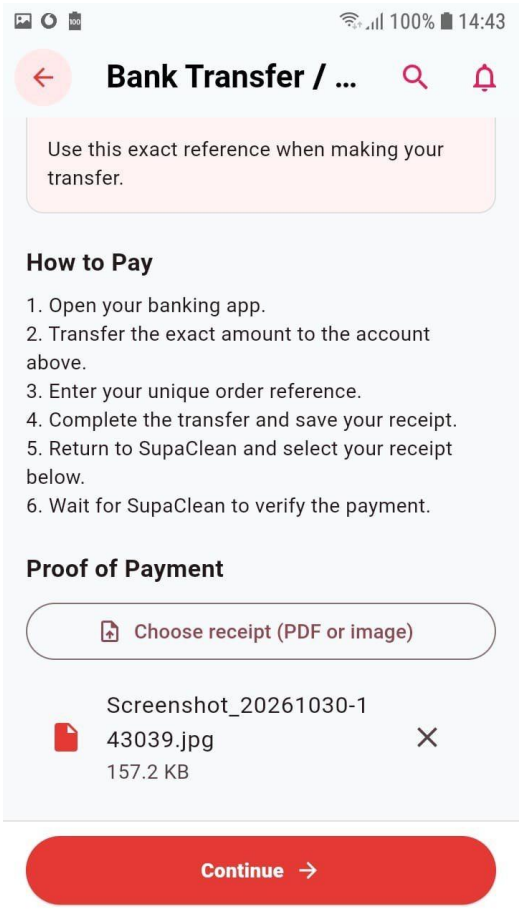
9. Upload Proof of Payment Screen

After making a bank transfer, the customer uploads a PDF proof of payment or pastes the bank confirmation text. The proof is submitted for verification, and the order's payment status can be updated after review.

Flutter screen — Payment Reference / Transfer



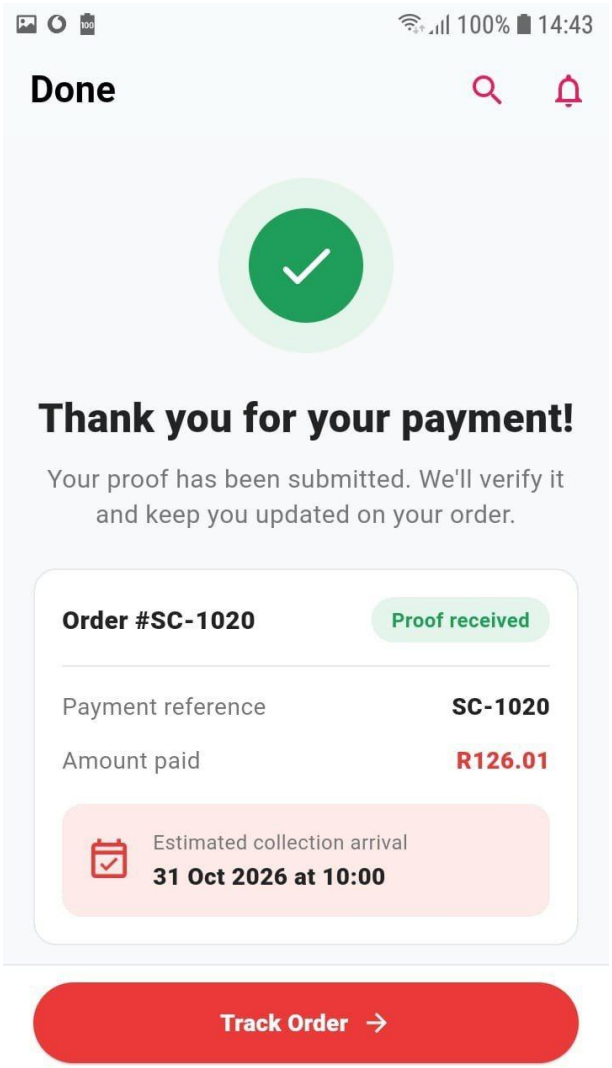
Flutter screen — Upload Proof of Payment



10. Payment Confirmation Screen

Confirms that the proof of payment has been received. It displays the order number, payment reference, amount and estimated collection or delivery time. Importantly, receiving proof does not necessarily mean the payment has been verified.

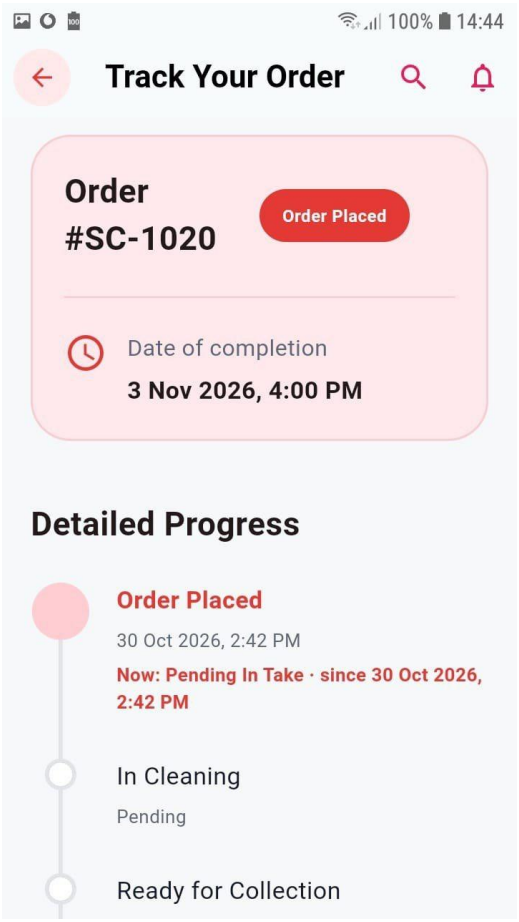
Flutter screen — Payment Confirmation



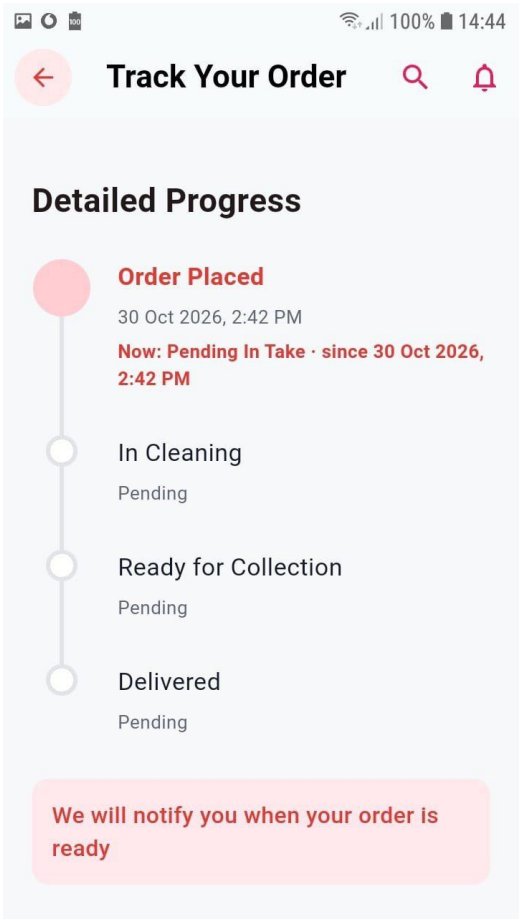
11. Track Your Order Screen

Displays the order's progress through stages such as Order Placed, Picked Up, In Cleaning, Quality Check, Ready for Collection and Delivered. Customers can see completed and pending stages and the estimated completion time.

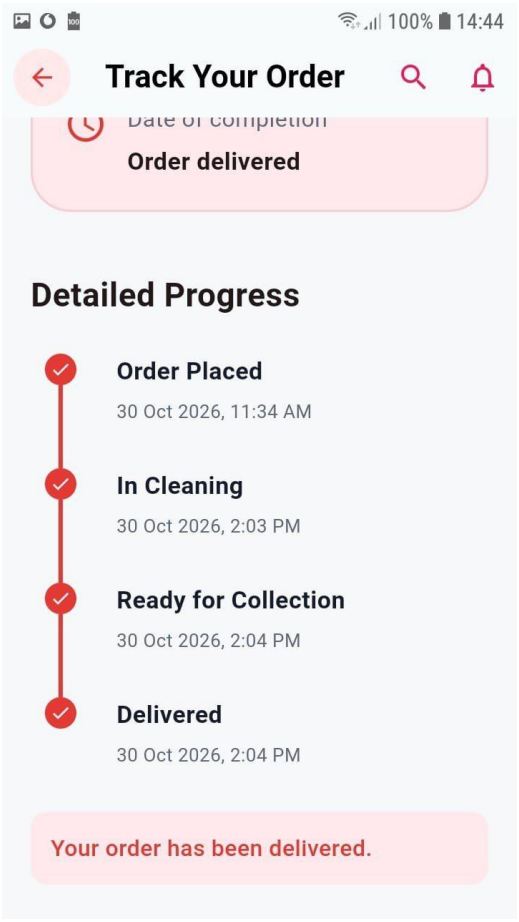
Flutter screen — Track Your Order



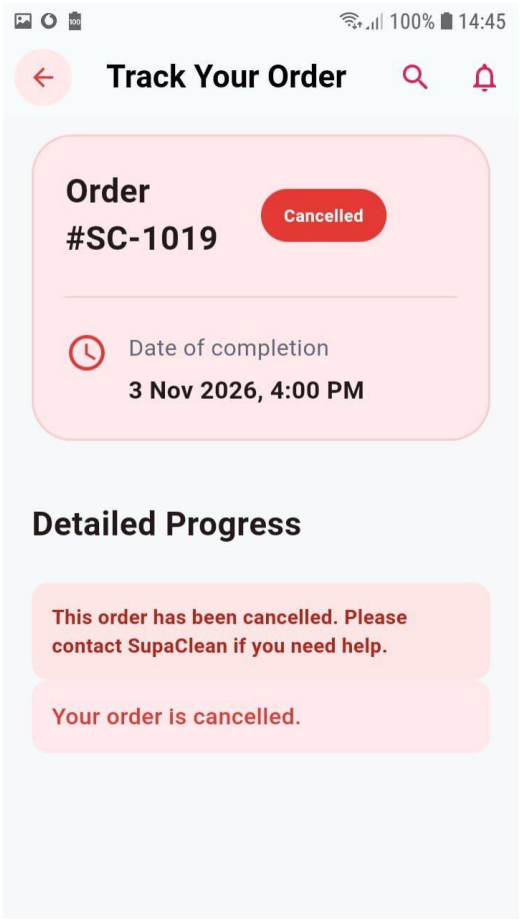
Flutter screen — Order Progress



Flutter screen — Delivered Order



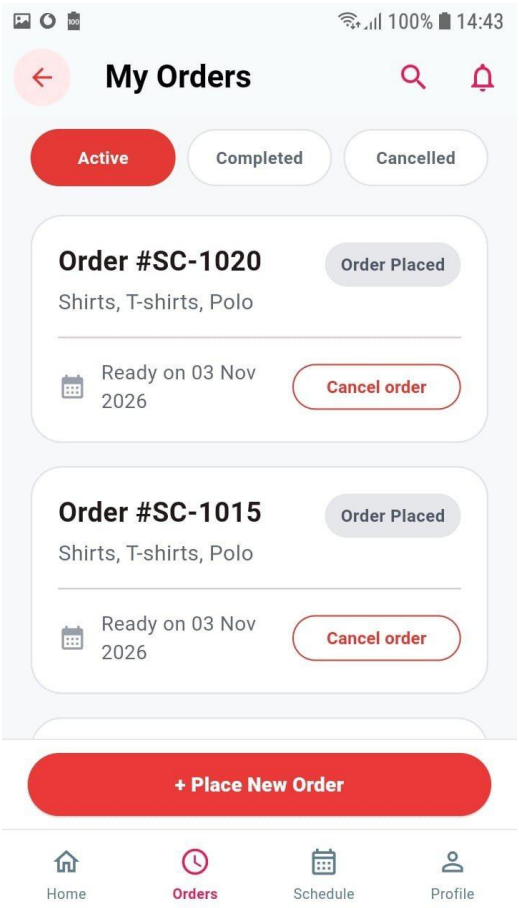
Flutter screen — Cancelled Order



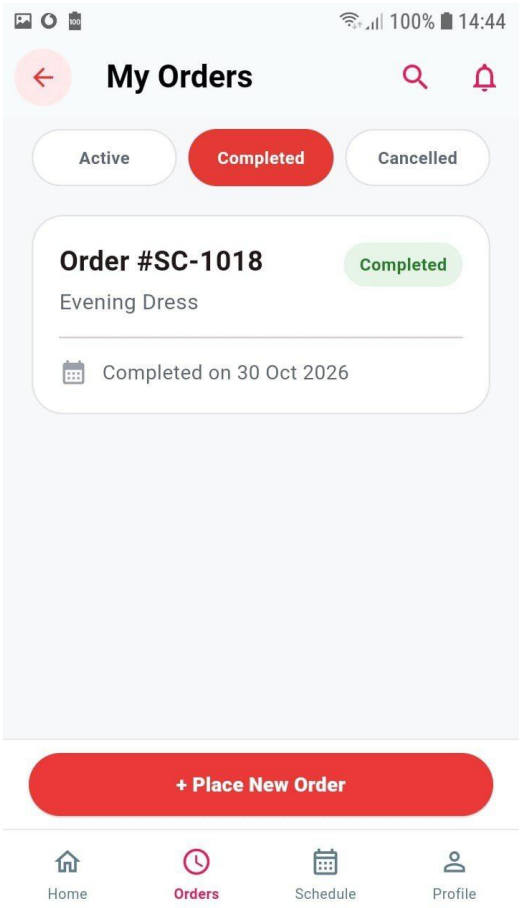
12. My Orders Screen

Lists the customer's active, completed and cancelled orders and orders history. Each order displays its reference, service type, status and expected completion date where applicable. Customers can use this screen to return to an order and check its progress.

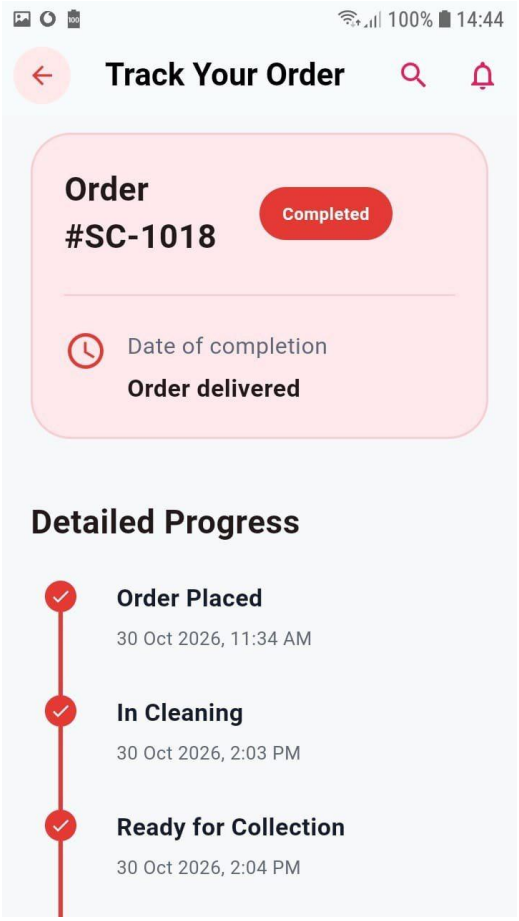
Flutter screen — My Orders



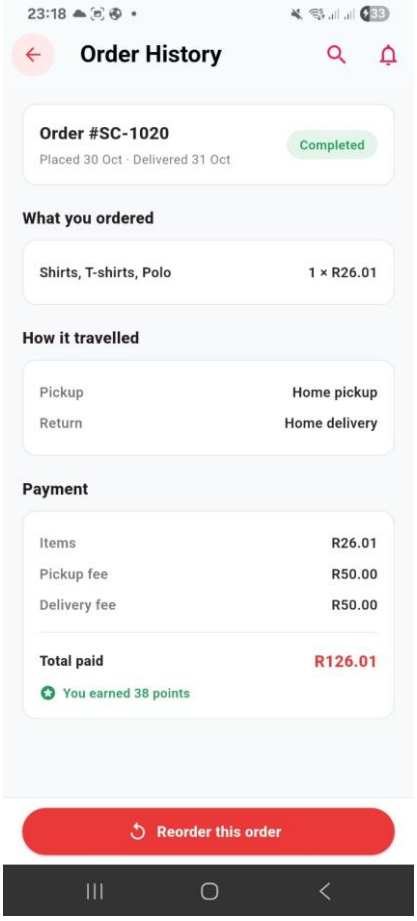
Flutter screen — My Orders / Order Status



Flutter screen — My Orders List



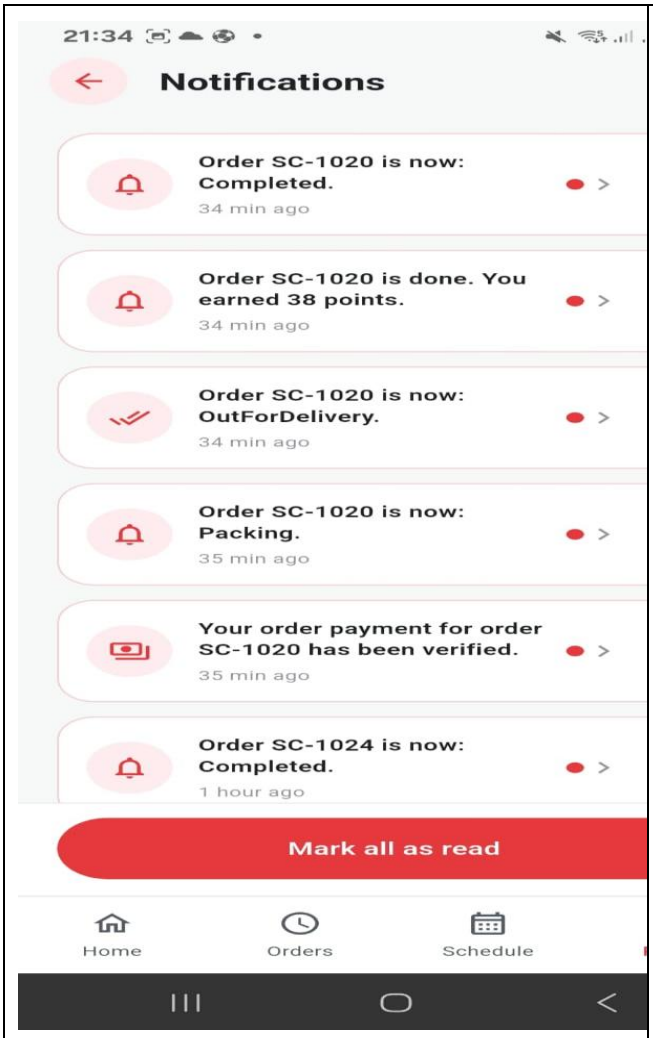
Flutter screen — My Orders History



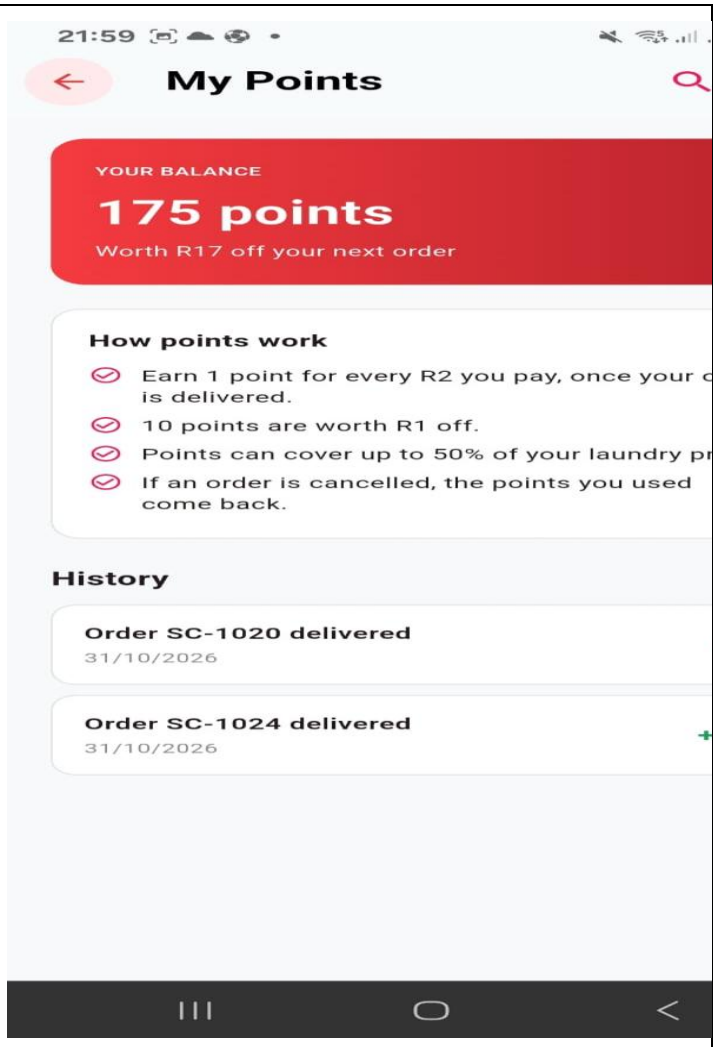
13. Notifications and Points Screen

Notifications: Keeps customers informed about important events, such as order confirmation, pickup, cleaning progress and delivery. Customers can mark notifications as read.

Points: Adds points when an order is delivered, can use points earned at payment and takes the discount off the total, returns points when order is cancelled and shows balance and points history



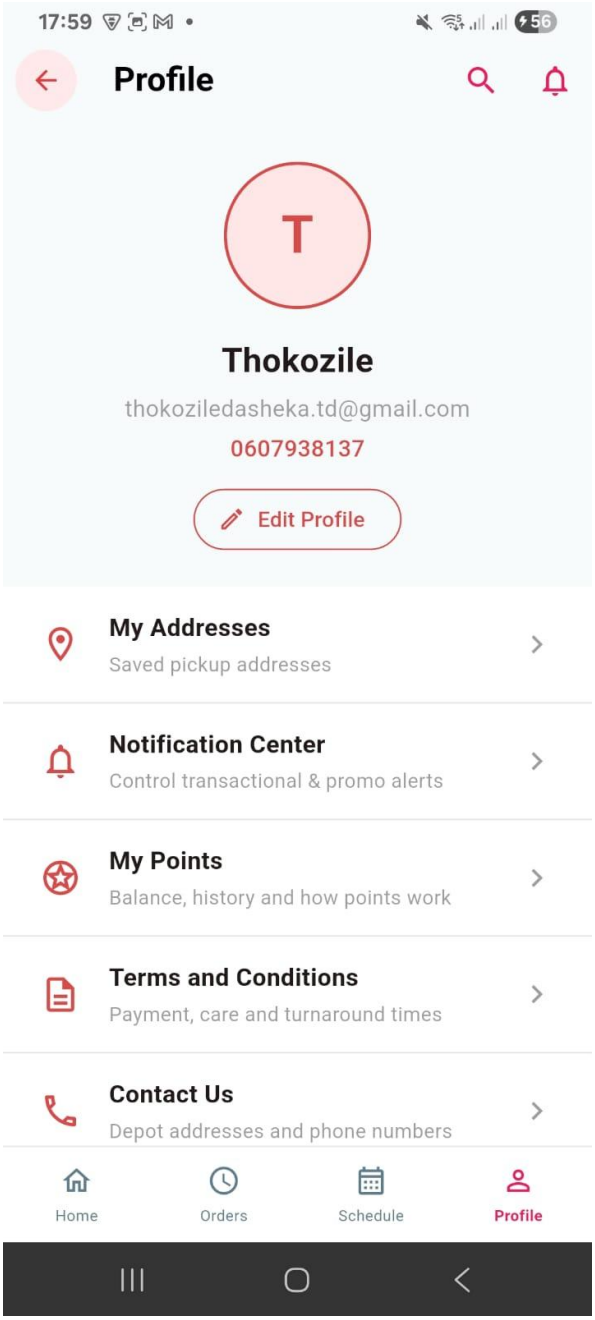
Flutter screen — Notifications



Flutter screen — Points

14. Profile Screen

Displays the customer's personal details and provides access to address management, payment methods, notification settings, help and support, and company information. The customer can also log out.



Flutter screen — Profile

3. How the screens work together

The workflow can be divided into four main stages:

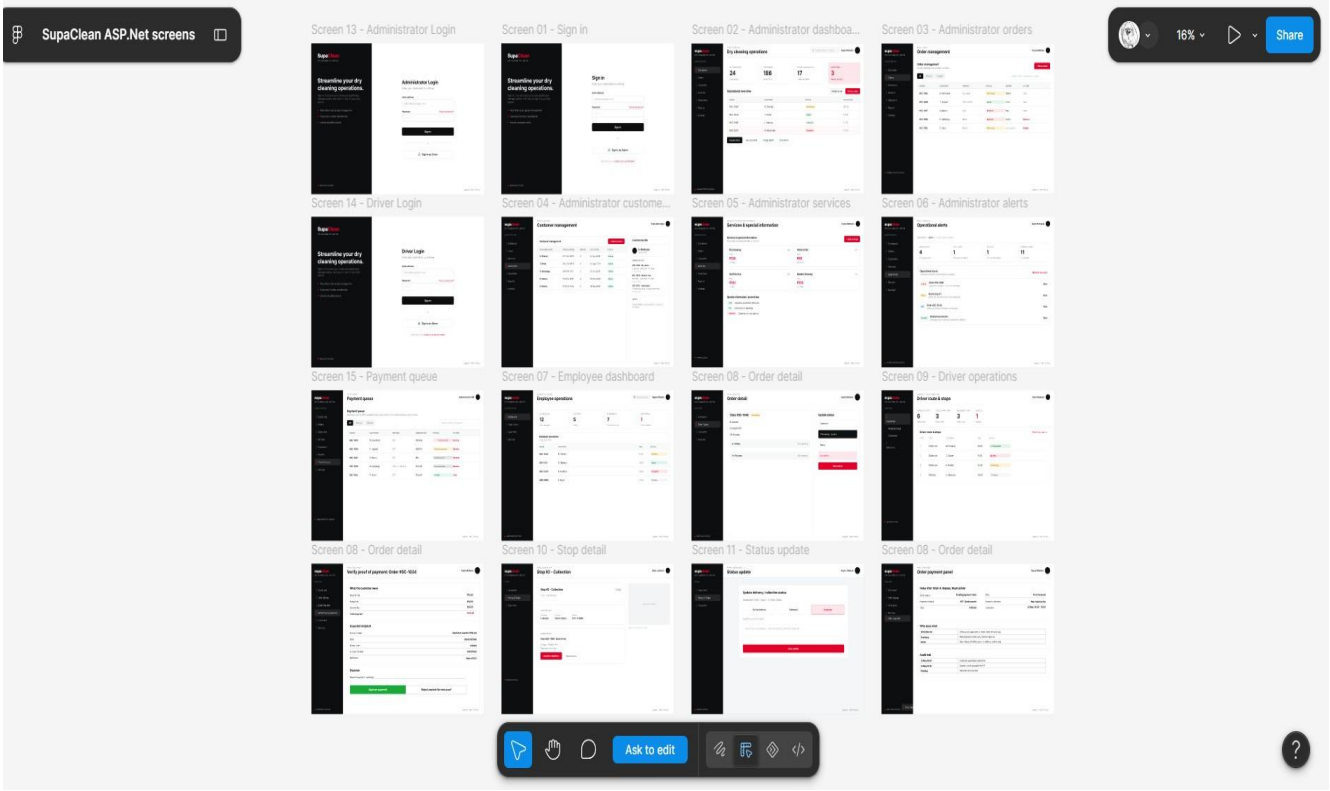
1. Account access: The customer opens the application and signs up or logs in.
2. Order creation: The customer explores the services, selects laundry items, specifies quantities and schedules pickup and delivery.
3. Payment processing: The customer reviews the total, chooses a payment method and, when paying by EFT, submits proof of payment for verification.
4. Order fulfilment: The customer monitors the order through tracking, My Orders and notifications until the laundry is ready and delivered.
5. Points: Adds points when an order is delivered, can use points earned at payment and takes the discount off the total, returns points when order is cancelled and shows balance and points history

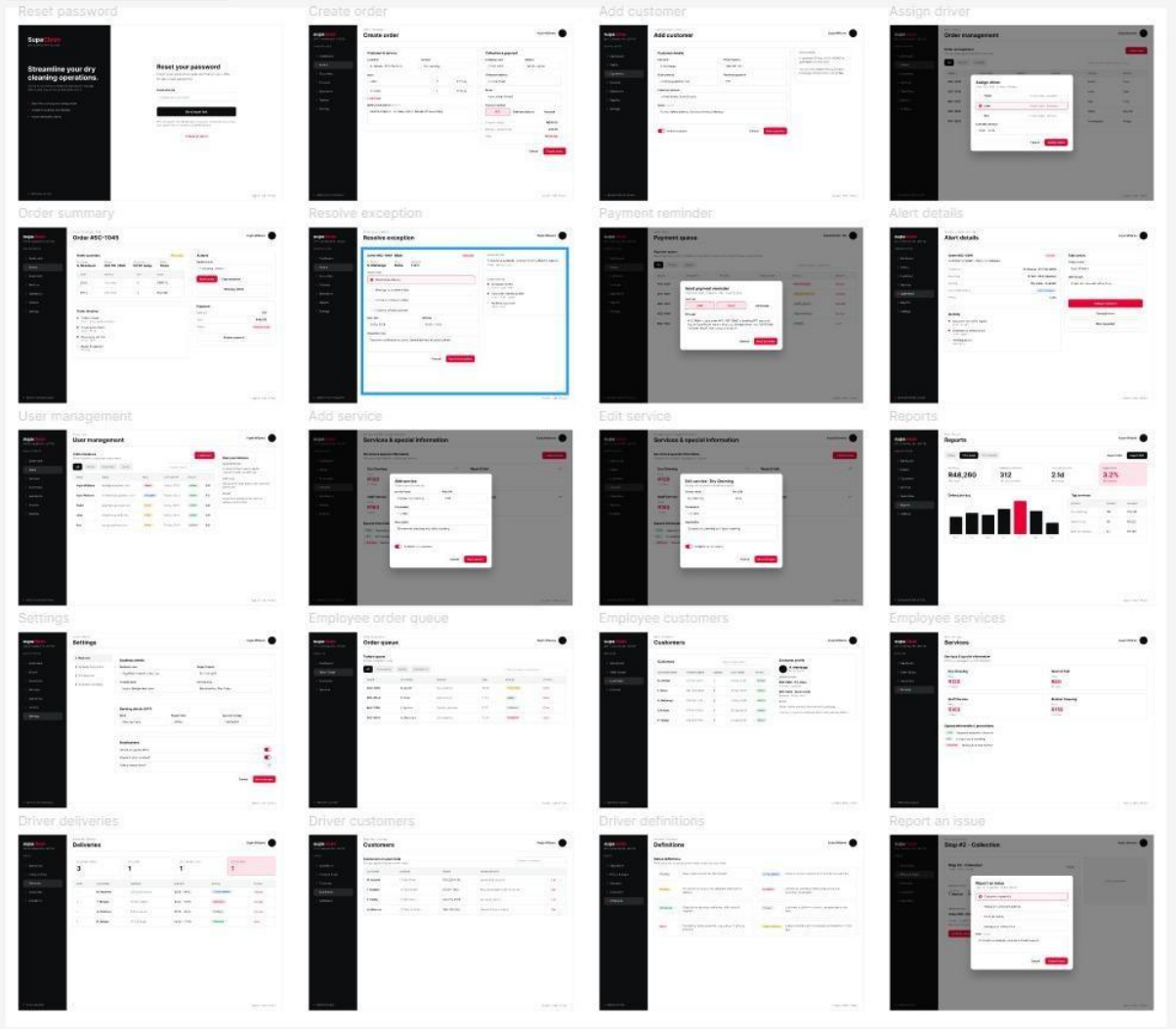
The bottom navigation bar provides access to the main sections, Home, Orders, Schedule and Profile; so, customers can move between these areas without restarting the process.

4. Important workflow considerations

- Payment verification: The application should distinguish between payment due, proof received, and payment verified.
- Order status updates: The tracking screen, My Orders screen and notifications should reflect consistent order statuses.
- Scheduling validation: The system should check that pickup and delivery dates, and time slots are valid.
- Price calculation: Service prices, quantities, pickup charges and delivery charges should be calculated consistently across the order and payment screens.
- Data consistency: The customer's selected services, address, schedule and payment details should remain associated with the same order throughout the workflow.

ASP.NET Web Application Screens





Overall application workflow

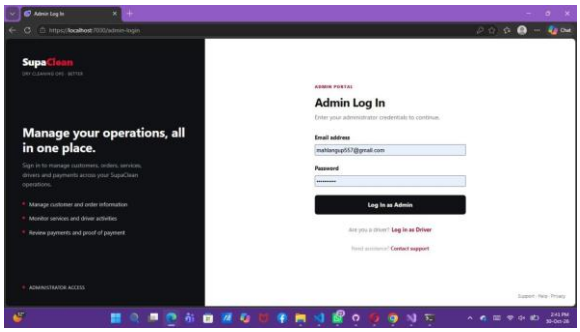
The flow begins with the login screen, where users access the functions available to them. Each screen connects to another stage of the dry-cleaning process, from creating an order to cleaning garments and delivering them to customers.

1. Login and access flow

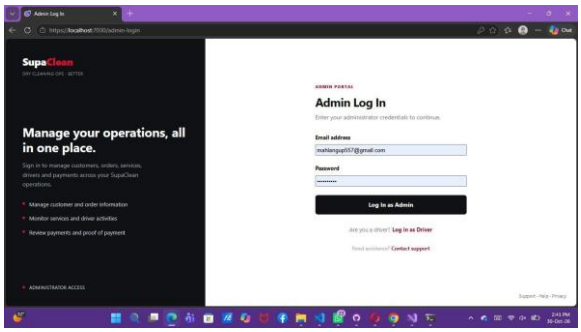
Screen 1: Login

Users enter their email address and password to sign in. The interface separates administrator, employee, and driver access. If a user forgets their password, they can select the password reset option and receive a reset link.

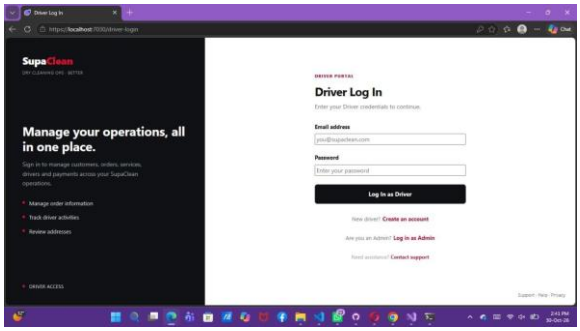
ASP.NET screen — Administrator Login



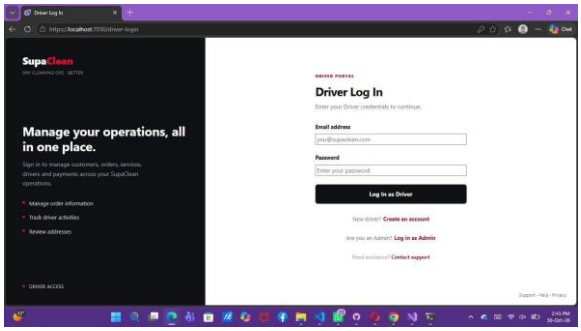
ASP.NET screen — Administrator Login



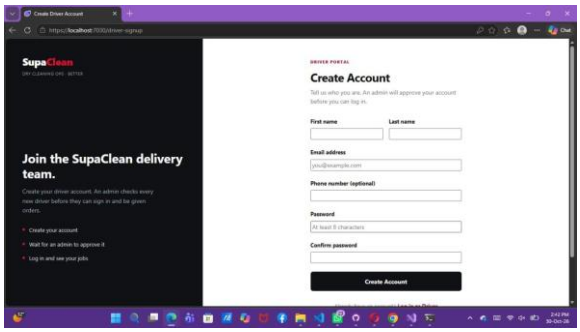
ASP.NET screen — Driver Login



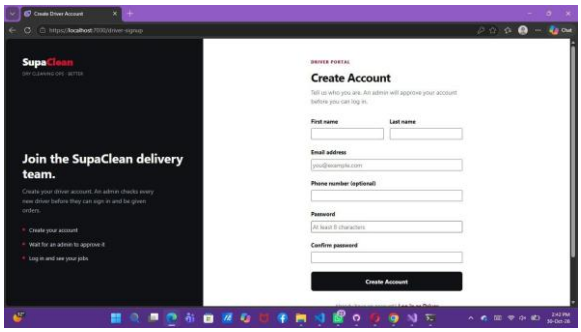
ASP.NET screen — Driver Login



ASP.NET screen — Driver Account Registration



ASP.NET screen — Driver Account Registration



Screen 2: Role-based dashboard

After signing in, users access the appropriate dashboard. Administrators oversee the whole operation, employees manage assigned orders, and drivers focus on collections and deliveries.

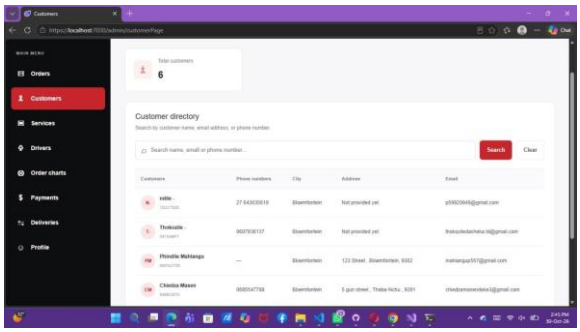
2. Administrator flow

The administrator controls the main operations of SupaClean. The administrator can create orders, manage customers, assign drivers, check payments, and monitor business performance. a. Dashboard

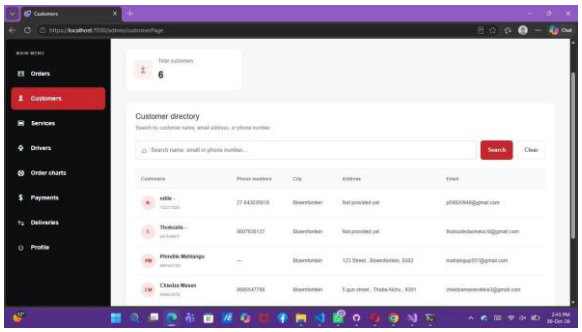
b. View orders, collections and alerts

c. Manage orders and customers

ASP.NET screen — Customer Directory



ASP.NET screen — Customer Directory

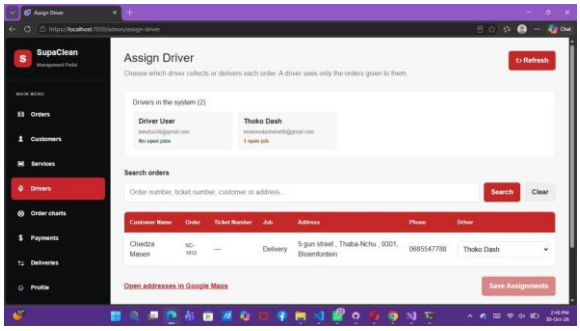


d. Create orders, register customers and select services

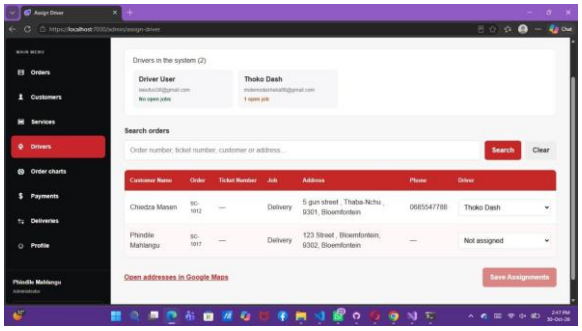
e. Assign a driver

f. Choose an available driver and collection time

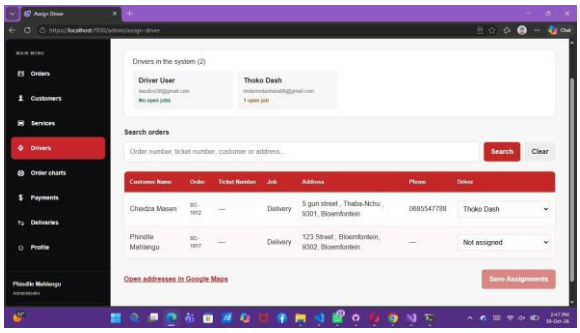
ASP.NET screen — Assign Driver



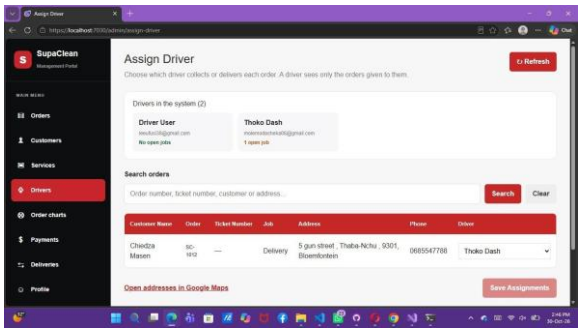
ASP.NET screen — Driver Assignment List



ASP.NET screen — Assign Driver



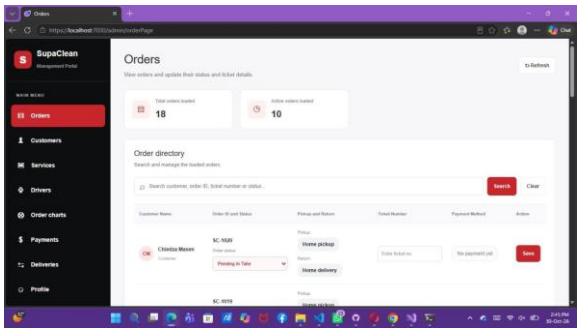
ASP.NET screen — Driver Assignment List



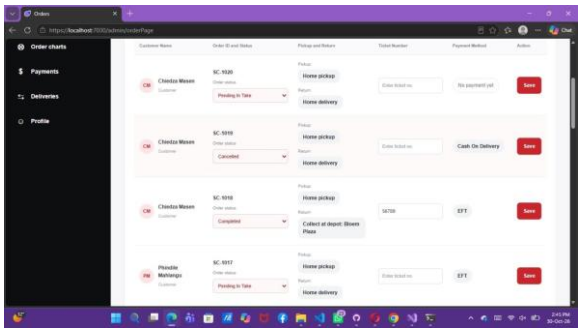
g. Monitor and manage the order

h. Track processing, readiness, delivery and payment The administrator can also:

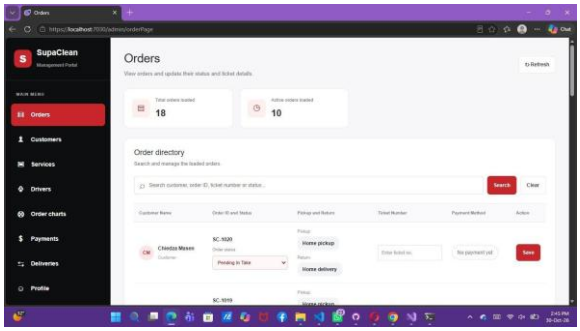
ASP.NET screen — Administrator Order Directory



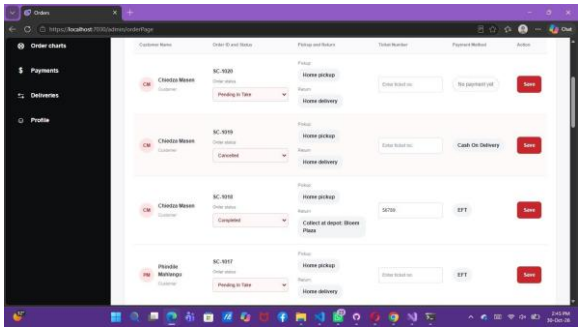
ASP.NET screen — Order Management Details



ASP.NET screen — Order Directory

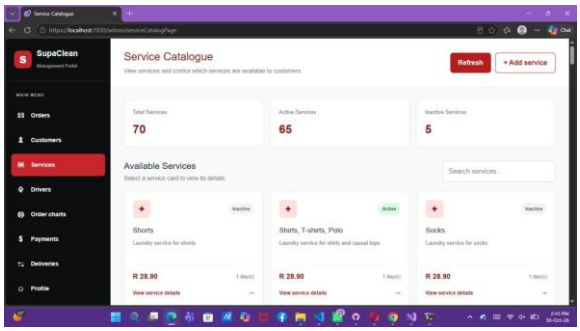


ASP.NET screen — Order Management Details

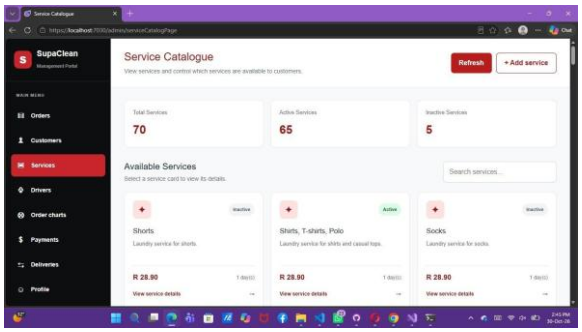


a. Manage services: Edit service prices, turnaround times and promotions.

ASP.NET screen — Service Catalogue

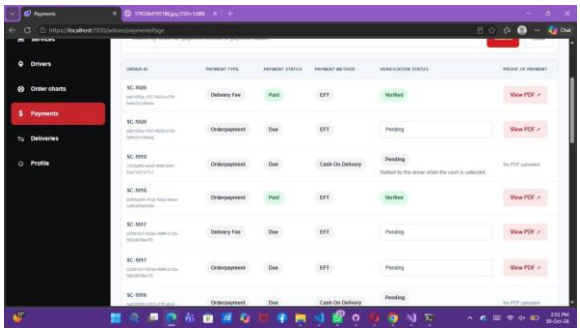


ASP.NET screen — Service Catalogue

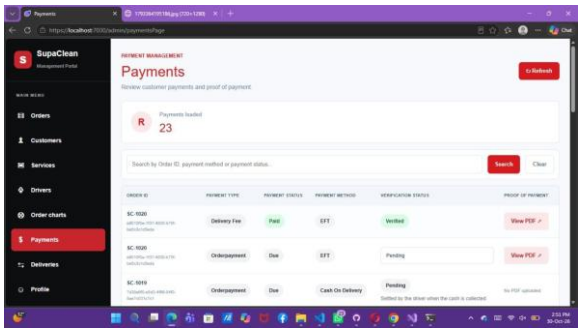


b. Manage payments: Review proof of EFT payment, approve or reject it, and send reminders.

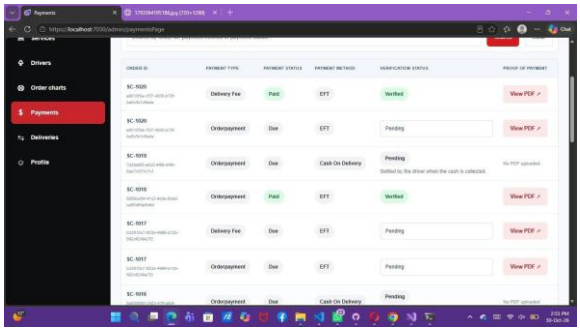
ASP.NET screen — Payment Verification Records



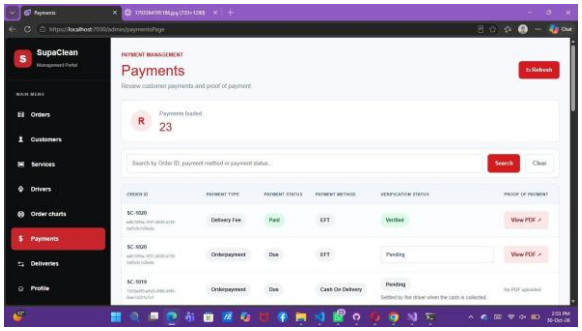
ASP.NET screen — Payments Management



ASP.NET screen — Payment Verification



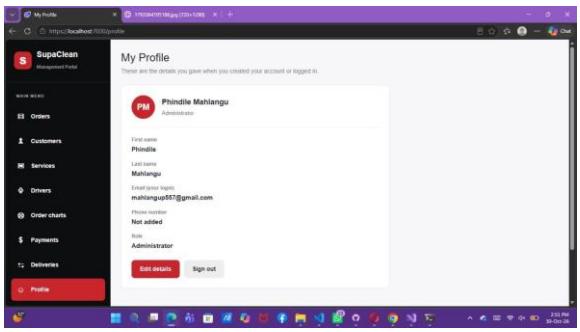
ASP.NET screen — Payments Management



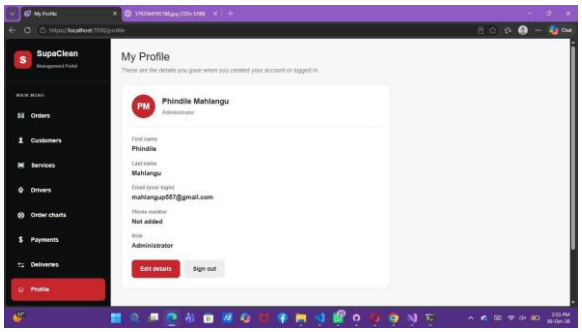
c. Resolve exceptions: Handle failed deliveries, customer complaints and rescheduling.

d. Manage the business: Control user roles, view reports, and update settings.

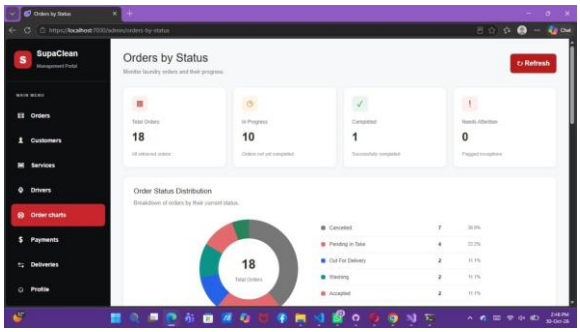
ASP.NET screen — Administrator Profile



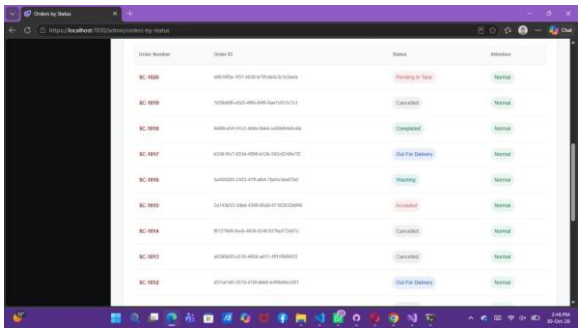
ASP.NET screen — Administrator Profile



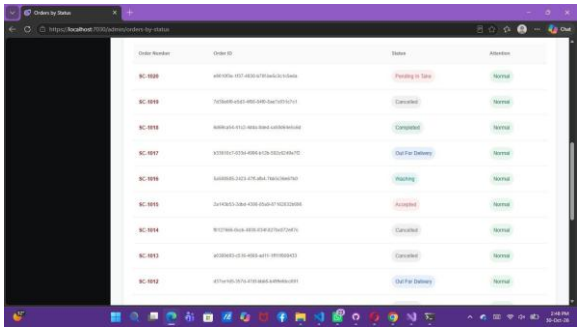
ASP.NET screen — Order Status Dashboard



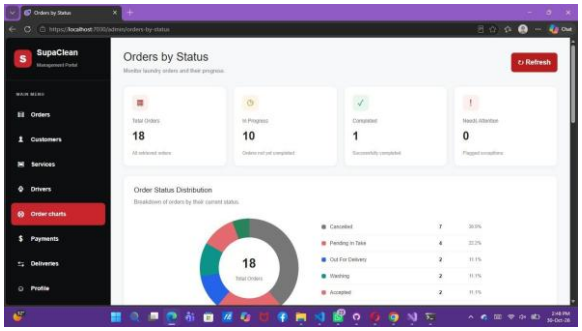
ASP.NET screen — Order Status List



ASP.NET screen — Order Status List



ASP.NET screen — Order Status Dashboard



3. Driver flow

a. Drivers use the prototype to manage their assigned collection and delivery routes. b. Driver dashboard

c. View assigned stops, deliveries and alerts

d. Open route and stop details

e. Check the customer's address, contact number and order

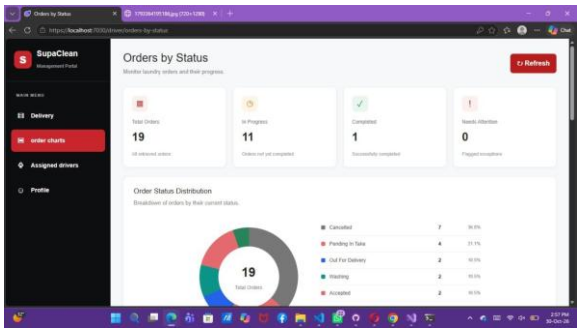
f. Perform collection or delivery

g. Confirm the stop and attach a photo when required

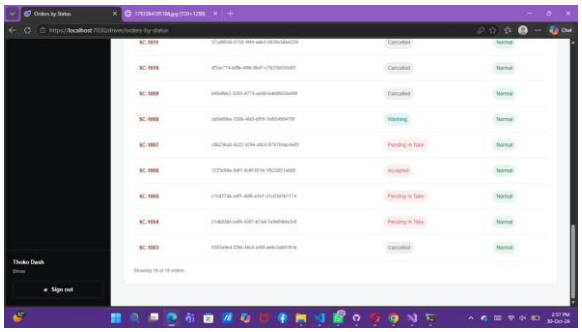
h. Update status or report an issue

i. Record completion or report problems such as an unavailable customer

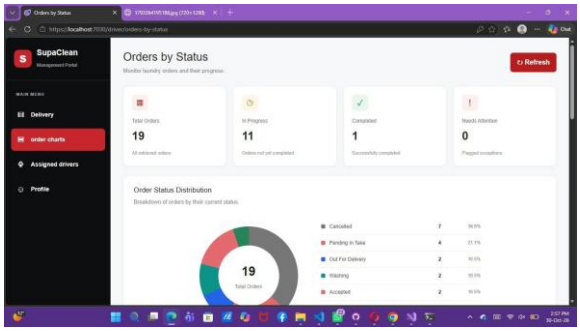
ASP.NET screen — Driver Order Status Dashboard



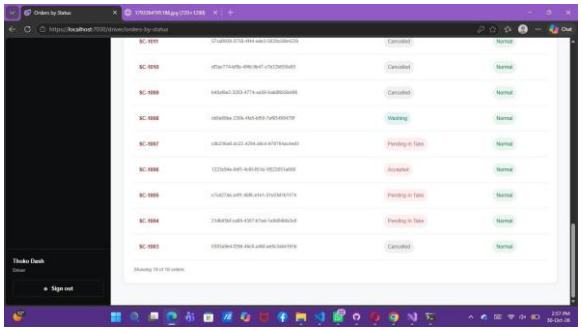
ASP.NET screen — Driver Order Status List



ASP.NET screen — Driver Order Status Dashboard

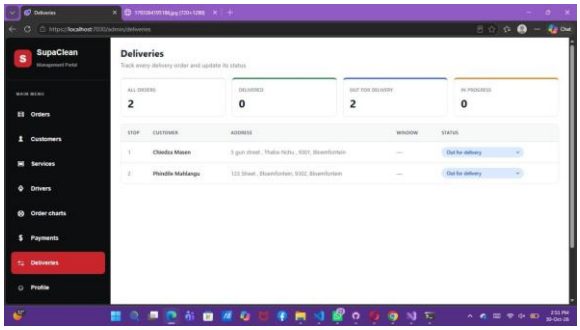


ASP.NET screen — Driver Order Status List

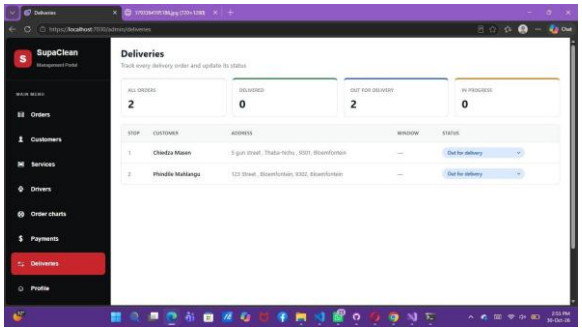


j. Drivers can view customers on their routes and their contact information. For cash-on-delivery orders, drivers can confirm payment at the doorstep.

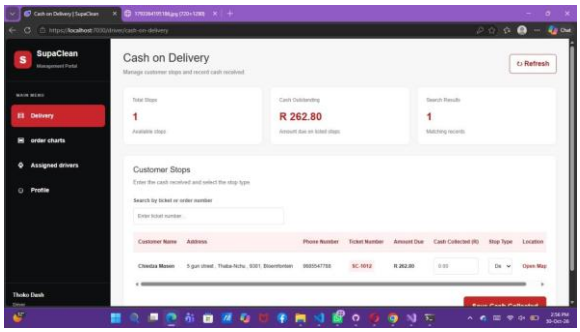
ASP.NET screen — Deliveries



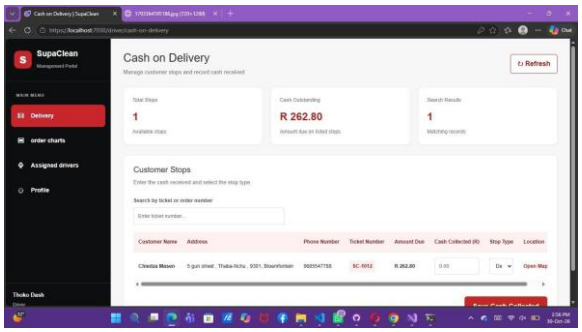
ASP.NET screen — Delivery Management



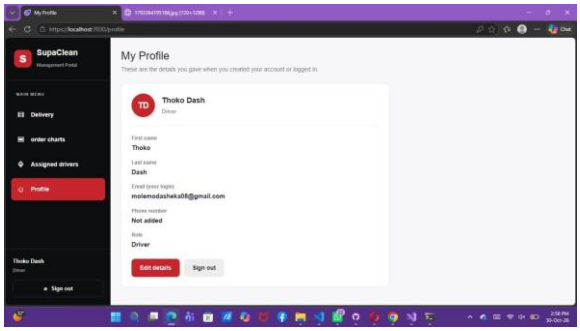
ASP.NET screen — Cash on Delivery



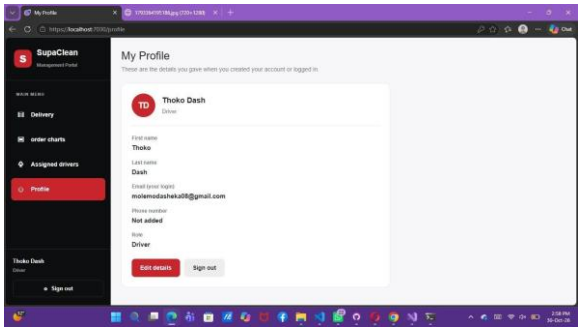
ASP.NET screen — Cash Collection



ASP.NET screen — Driver Profile



ASP.NET screen — Driver Profile



4. Payment and exception flows

These are supporting flows that connect the main screens and help the business handle problems.

Payment flow

a. The customer pays by EFT outside the application.

b. The customer submits proof of payment.

c. The administrator checks the amount and proof.

d. The administrator approves or rejects the payment.

e. The system records the payment status and audit trail.

Exception flow

a. A driver or another user reports a problem.

b. The system records the exception and alerts the administrator.

c. The administrator investigates and selects a resolution.

d. The order may be rescheduled, reassigned, collected in-store, or resolved through a credit or refund.

5. Overall prototype flow

I. Login

II. Select role: Administrator, Employee or Driver III. Administrator

IV. Create and assign orders

V. Employee

VI. Process and update orders

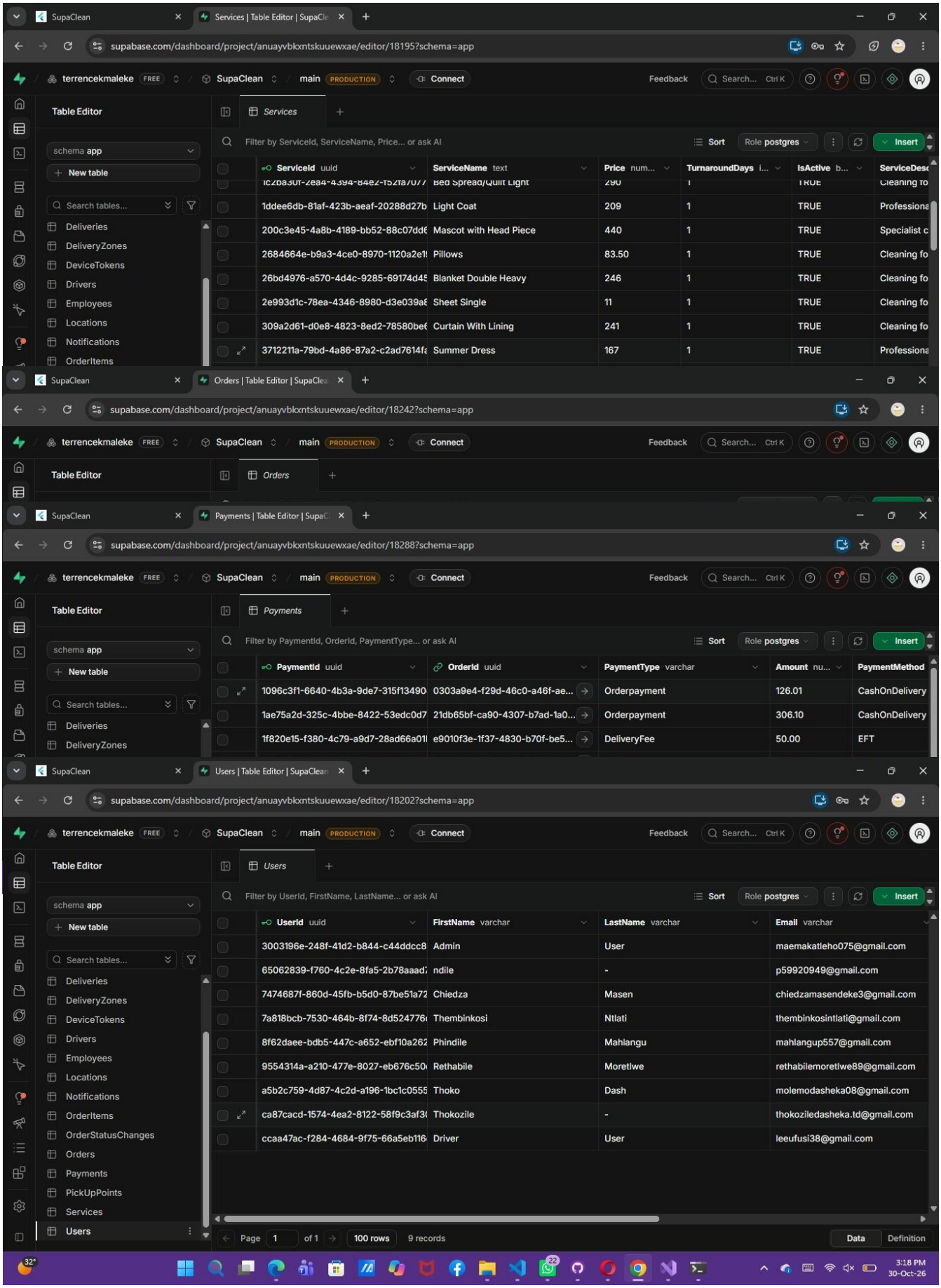
VII. Driver

VIII. Collect and deliver

IX. Order completion and status tracking

X. Payment verification, exception handling and reporting

Shared backend and Integration evidence (Supabase)



Implementation Changes and Remaining Limitations

During the implementation of the SupaClean system, certain refinements were made to align the applications with the approved requirements and improve usability.

Approved changes and implementation refinements:

• Administrative functions were consolidated into the ASP.NET web application. The Flutter mobile application is intended for customers, while drivers and administrators use the web application to manage operational activities.

• The existing ticket/docket process was retained. The system improves digital customer interaction and order information management rather than replacing the business's existing operational process.

• The mobile interface was refined to provide consistent navigation and a more user-friendly experience across the service, order, notification and profile screens.

• The Flutter mobile application, ASP.NET web application and Supabase backend were successfully integrated. Data can be shared between the applications through the backend, supporting the system's connected workflows.

Remaining limitations:

• Live driver GPS tracking is not included. Customers can view recorded order-status updates instead.

• An online payment gateway is not included. Customers use external payment methods such as EFT/bank transfer or cash, with payment records or proof handled through the web application.

• The system continues to support the existing ticket/docket process rather than fully replacing it.

• Additional features outside the approved project scope may be considered in future development.

These changes and limitations maintain the approved project scope while allowing the team to deliver an integrated solution consisting of the Flutter mobile application, ASP.NET web application and shared Supabase backend.

Systems Workflows and Shared Processes

Example A: Customer places an order

1. Customer — Flutter

Selects a service, enters item details and submits the order.

2. Supabase backend

Validates and stores the order through the configured application connection and database rules.

3. ASP.NET web application

An authorised employee retrieves and views the saved order.

4. Final result

The order appears in the administrative interface and is associated with the correct customer.

Example B: Administrator updates order status

1. Employee action

An authorised employee selects a new status in ASP.NET.

2. Backend update

ASP.NET submits the change to Supabase, where it is saved if the operation succeeds.

3. Customer application

Flutter retrieves the latest order status or receives an update through a configured real-time subscription.

4. Final result

The customer sees the updated status in the relevant order screen.

Example C: External payment and verification

External payment and verification workflow:

1. *For an EFT/bank transfer, the customer views the company's bank details and the unique payment reference associated with the order, then makes the transfer through their own banking channel outside the application.*
2. *After completing the transfer, the customer returns to the Flutter application and uploads the proof of payment, where this upload function is available in the implemented prototype.*
3. *The proof of payment is stored and associated with the relevant customer, order and payment record. Submission of proof means that evidence has been received; it does not mean that the payment has already been verified.*
4. *An authorised administrator reviews the payment amount, reference and submitted proof through the ASP.NET web application.*
5. *The administrator records the verification outcome. The payment record is updated as approved/verified or rejected/awaiting correction, as applicable.*
6. *For cash on delivery, the customer pays when the order is delivered. The authorised driver or staff member records the collection of cash through the web application, subject to the permissions configured for that role. No online payment gateway is used.*

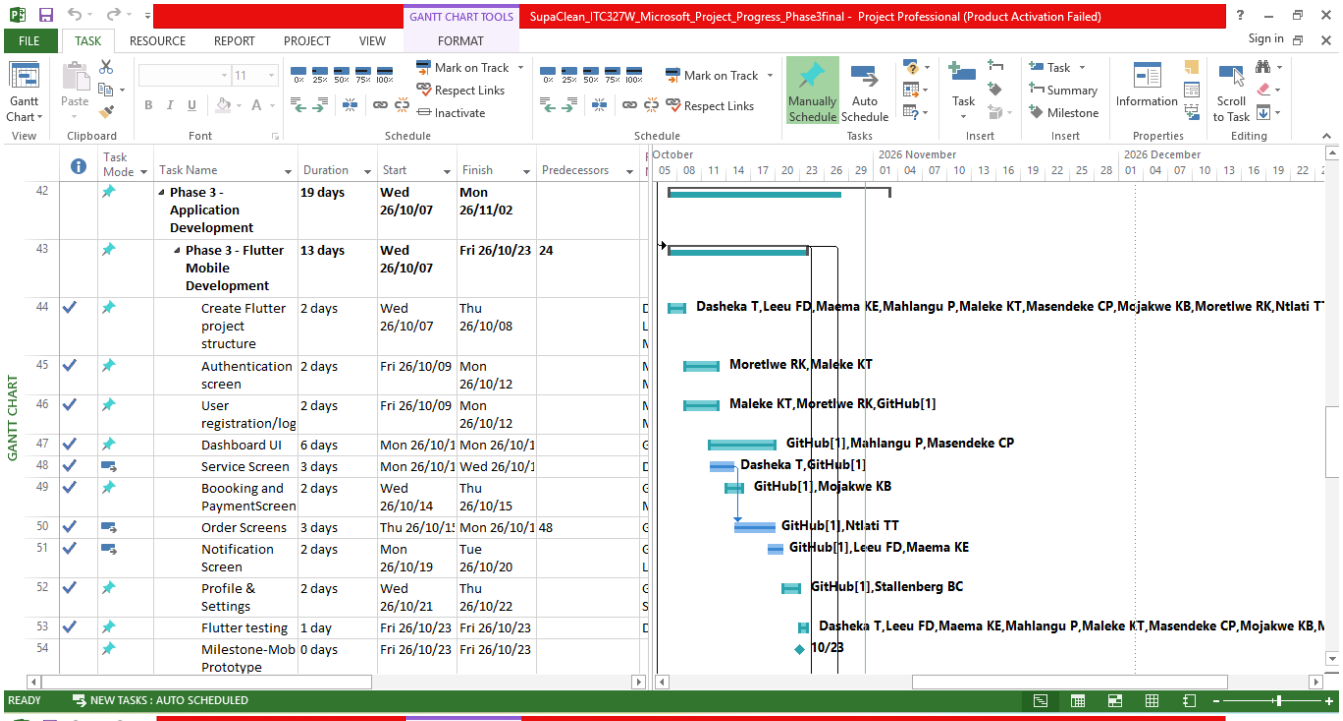
Final Prototype Summary

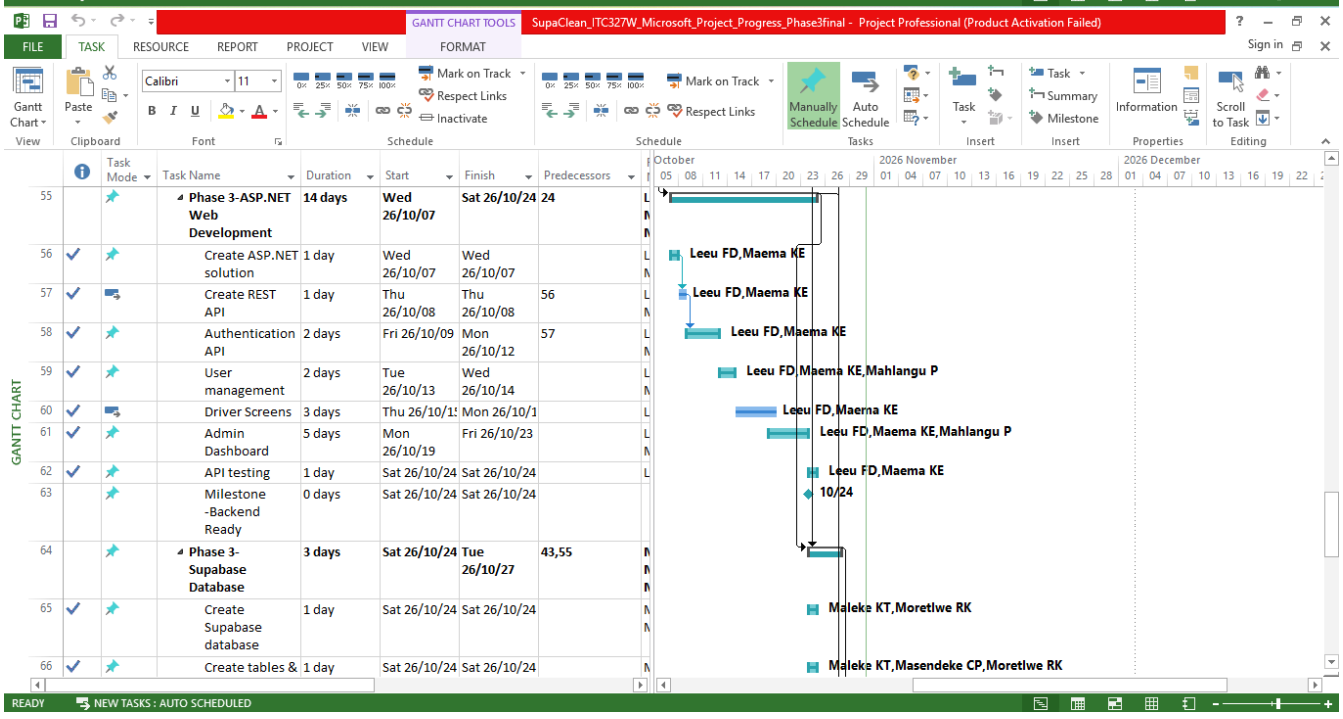
The final SupaClean prototype brings together the Flutter customer mobile application, the ASP.NET administrative web application and the Supabase backend. The prototype is intended to improve the management of laundry orders and customer information while supporting the approved project requirements.

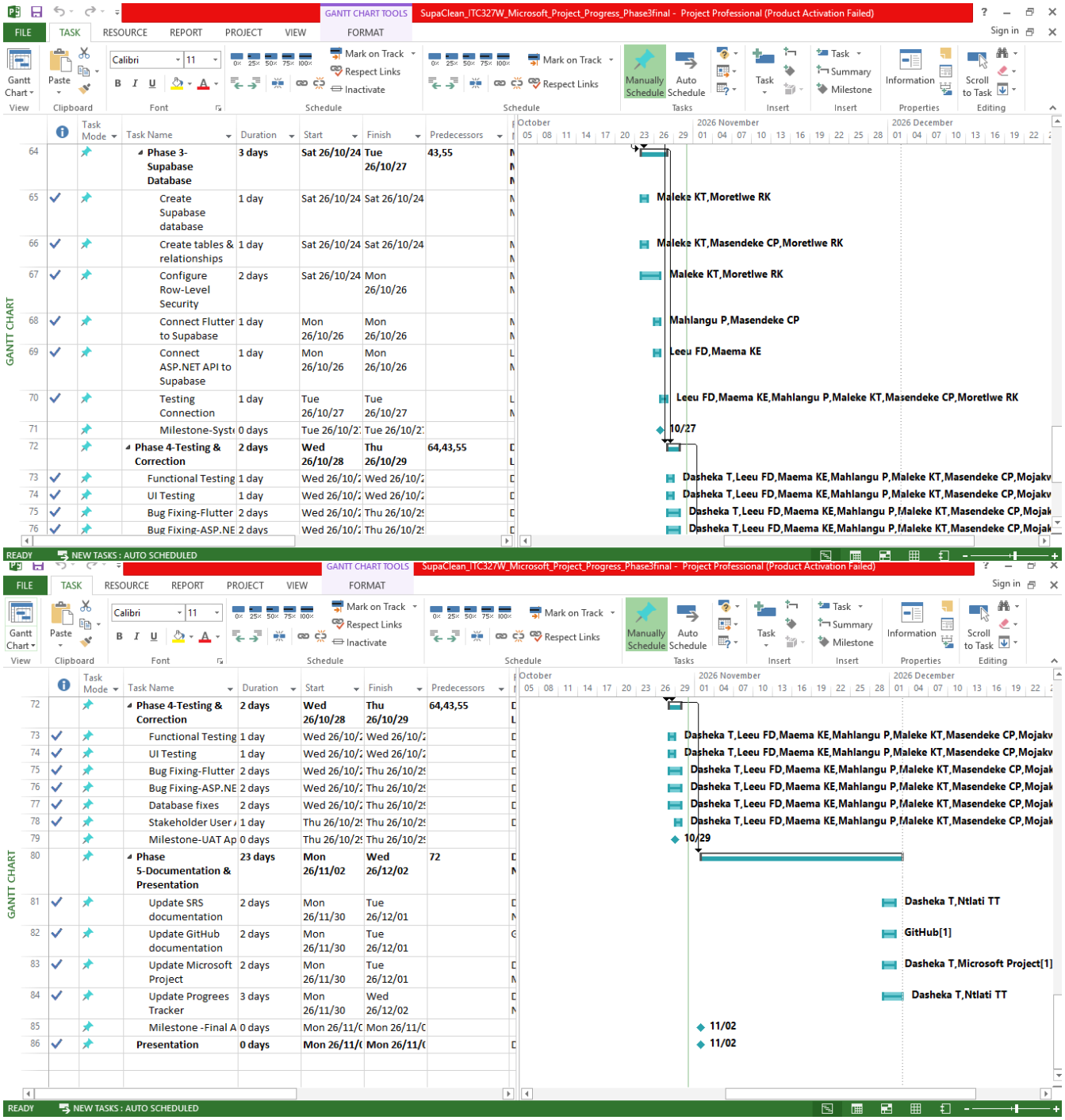
The implementation evidence presented in this section identifies the functions that have been demonstrated, the integration between the system components, and the remaining incomplete functions or known limitations.

The final implementation status must be supported by the functions demonstrated in the prototype and the evidence included in this document. A screen design alone does not prove that a function is operational. The group should mark a requirement as Working only when the relevant workflow has been tested and can be demonstrated; otherwise, identify it as Incomplete, Not implemented or To be verified. The final prototype remains within the approved project scope, excluding live driver GPS tracking and online payment gateway integration.

Project Plan – Phase 3







33. Conclusion

The Phase 3 document describes the SupaClean system as a connected solution comprising a Flutter customer mobile application, an ASP.NET staff and management web application, and a shared Supabase backend. The project focuses on improving customer interaction and managing customer and order information while supporting the existing ticket/docket process. The system supports service selection, pickup and delivery scheduling, order-status information, notifications and external payment-method workflows. Live driver GPS tracking and online payment gateway integration are excluded. The diagrams and implementation evidence should be checked against this scope before final submission.

AI usage

AI tools were used as supporting resources during the project to help the group understand requirements, interpret the assessment rubric, organise the document and improve wording, grammar and structure. The group reviewed and adapted AI-assisted suggestions against stakeholder feedback, the agreed requirements and the actual project work. The group remains responsible for the accuracy of the requirements, designs, implementation claims and submitted work.

Supporting Evidence and Appendices

Appendix Documents

- Appendix A – Stakeholder Interview Questions and Responses | PDF:

SupaClean_Appendix_A_Stakeholder_Interview.pdf

Appendix B- Feasibility Study

SupaClean_Appendix_B_Feasibility_Study.pdf

- Appendix C – Evidence of Stakeholder Engagement | PDF:

SupaClean_Appendix_C_Stakeholder_Engagement.pdf

- Appendix D – Stakeholder Confirmation of the Findings | PDF:

SupaClean_Appendix_D_Stakeholder_Confirmation.pdf

Appendix E- Risk Register

SupaClean_Appendix_E_Risk_Register.pdf

Appendix F – Class Diagram

SupaClean_UML_Class_Diagram_Explanation.pdf

Appendix G -Stakeholder profile

SupaClean_Stakeholder_Profile.Docx